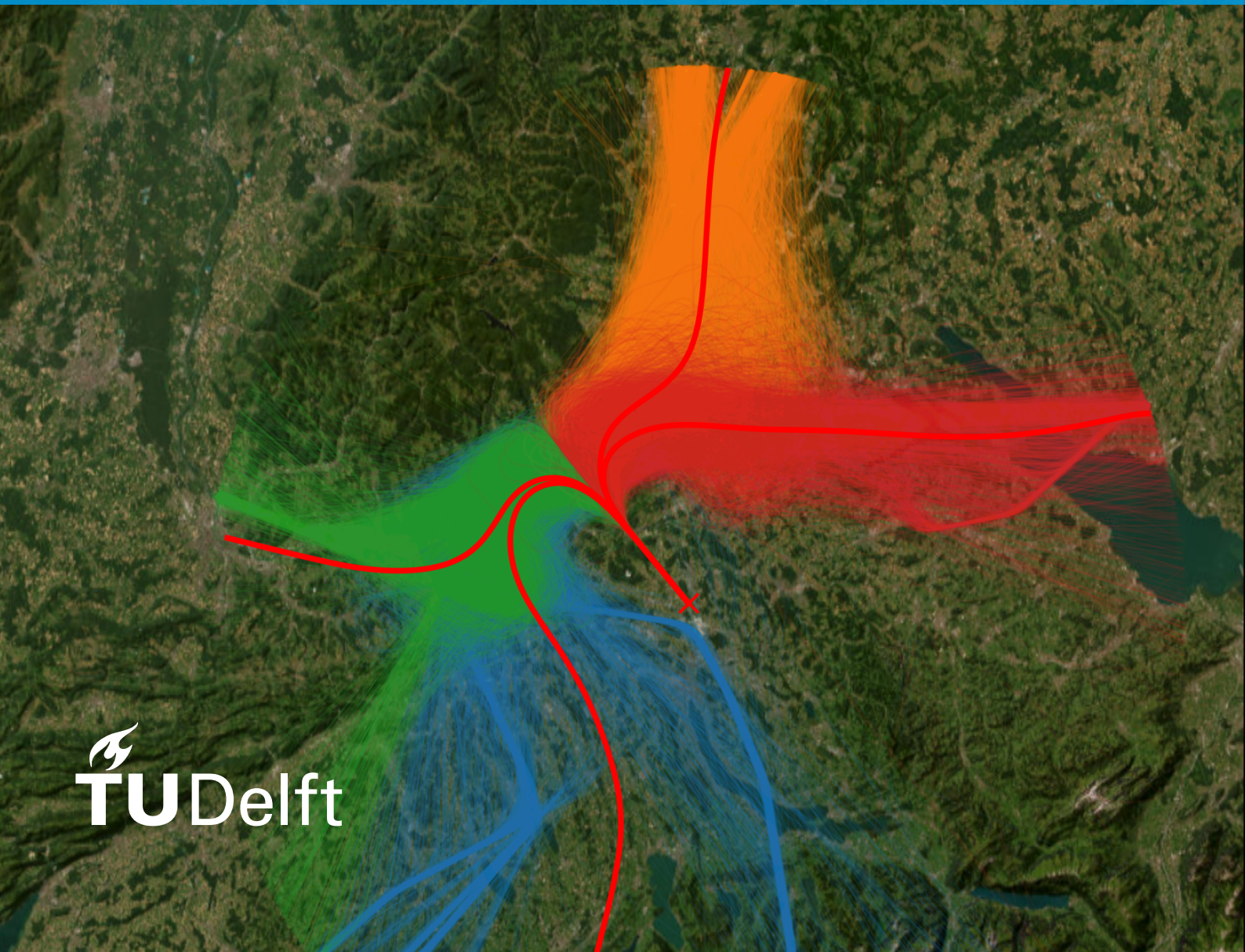


Generation of TMA Landing Trajectories Using Diffusion Models in Kinematic State Space

Master of Science Thesis

Gerard Martínez Vilella



Generation of TMA Landing Trajectories Using Diffusion Models in Kinematic State Space

Master of Science Thesis

Thesis report

by

Gerard Martínez Vilella

to obtain the degree of Master of Science
in Aerospace Engineering

Thesis committee:

Chair:	Dr. O.A. Sharpan's'kykh
Supervisors:	Ir. K.Y.S.B. Aly
Place:	Faculty of Aerospace Engineering, Delft
Project Duration:	February, 2026 - July, 2026
Student number:	6578128

An electronic version of this thesis is available at <http://github.com/MVile02/PI-DM>.

Abstract

As Air Traffic Management (ATM) systems face escalating complexity, the integration of Artificial Intelligence and Machine Learning has become critical for operational safety and efficiency. However, the development of these advanced tools is severely bottlenecked by the limited availability of comprehensive operational datasets due to privacy constraints and the high costs of data acquisition. Synthetic Data Generation (SDG) offers a promising solution to this data access problem, yet existing methodologies struggle to balance physical realism with operational diversity. Model-driven approaches fail to capture human-induced variance, while traditional data-driven models, such as Generative Adversarial Networks (GANs), frequently suffer from severe mode collapse and training instability.

To address these critical limitations, this thesis proposes a novel Conditional 1D U-Net Diffusion Model explicitly tailored for the generation of continuous 4D aircraft terminal trajectories. Operating directly within a deterministic Kinematic State Space, the streamlined 1D architecture natively processes high-frequency time-series data without the computational overhead of over-parameterized models adapted from image generation. Furthermore, the framework integrates Feature-wise Linear Modulation (FiLM), establishing a highly steerable methodology capable of effectively conditioning trajectory synthesis on specific operational requirements, such as aircraft type.

Trained on empirical arrival trajectories for Zurich Airport (LSZH) Runway 14, the model’s performance was evaluated using a rigorous, multi-dimensional statistical framework assessing spatial distribution, temporal fidelity, downstream predictive utility, and a data memorization check. The comparative analysis demonstrates that the proposed diffusion framework achieves highly competitive, state-of-the-art statistical results while fundamentally overcoming the training instability and mode collapse endemic to GAN-based architectures. Ultimately, this research provides a robust, computationally stable, and kinematically realistic foundation for generating high-fidelity, diverse synthetic air traffic datasets, offering a powerful open-source tool for airspace simulation, capacity analysis, and advanced algorithmic training in aviation.

Contents

List of Abbreviations	vi
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement.	1
1.3 Research Scope and Objective	2
1.4 Thesis Outline	2
I Literature Study & Research Definition	3
2 Literature Review and Theoretical Background	4
2.1 Synthetic Data: Definition and Overview	4
2.2 Representation of 4D Aircraft Trajectories	5
2.3 Temporal Dynamics and Sequence Generation.	5
2.4 Available Open Data Sources	8
2.5 Traditional Trajectory Generation Methods	9
2.6 Data-Driven Methodologies: Deep Generative Models	13
2.7 Evaluation of Synthetic Trajectories	16
3 Research Proposal	17
3.1 Introduction	17
3.2 Identified Research Gaps	17
3.3 Research Objective and Questions	18
3.4 Proposed Methodology and Work Packages	18
3.5 Expected Contributions	20
II Methodology	21
4 Methodology	22
4.1 Data Extraction and Preprocessing	22
4.2 Spatial Filtering and Clustering	24
4.3 Generative Architecture: 1D U-Net in Kinematic Space	25
4.4 Diffusion Model Training	29
4.5 Evaluation Framework	32
III Results & Discussion	36
5 Results	37
5.1 Trajectory Generation Results.	37
5.2 Statistical evaluation.	40

5.3	Comparison with Reference Models	42
6	Discussion	44
IV	Closure	46
7	Conclusions	47
8	Future Work	48
8.1	Assessment of Zero-Shot Generation Capabilities	48
8.2	Integration of Physics-Informed Regularization	48
8.3	Validation with BlueSky Simulator	48
8.4	Integration of Weather Conditionals and Congestion Scenarios	48
A	Software User Guide & Repository Structure	50
A.1	Code Repository Structure	50
A.2	Software User Guide	50
B	Benchmarking	52
B.1	Training Performance	52
B.2	Sampling Times	52
	References	56

Nomenclature

List of Symbols

c	Conditioning parameters vector	t	Time-step
$\mathcal{D}$	Dataset	T	Total flight duration
D	Discriminator neural network function	T_{diff}	Total number of timesteps
$D^2(F_X, F_Y)$	Squared Energy Distance between distributions F_X and F_Y	$v_{g,t}$	Groundspeed at a time-step
D_{KL}	Kullback-Leibler divergence	$\mathbf{v}$	Three-dimensional speed vector
dw	Standard Brownian motion (Wiener process)	$\mathcal{W}$	Standard Gaussian noise
$\mathbb{E}$	Expectation operator	$\mathbf{W}_t$	Standard three-dimensional Brownian motion
G	Generator neural network function	$\mathbf{x}_t$	Aircraft state at a time-step
h_t	Altitude at a time-step	x_0	Real uncorrupted data sample
$\mathbf{I}$	Identity matrix	$x_{T_{diff}}$	Pure Gaussian noise sample
K	Number of clusters	Z	Latent space
$\mathcal{L}$	Loss function term	α_t, β_t	Noise schedule variance parameters
L	Sequence length	ϵ	Exploration coefficient
N	Number of samples / Batch size	η	Learning rate parameter
$\mathcal{N}$	Standard Gaussian distribution	γ	Sub-loss weighting parameters
N_{real}	Number of real trajectories in the empirical dataset	μ	Mean of a Gaussian distribution
N_{synth}	Number of synthetic trajectories generated by the model	ψ_t	Track angle at a time-step
p	Reverse diffusion process (Decoder)	Σ	Covariance matrix of a Gaussian distribution
q	Forward diffusion process (Encoder)	σ_t	Squared correlation matrix
		θ	Model parameters vector

List of Abbreviations

ADS-B	Automatic Dependent Surveillance-Broadcast	LDM	Latent Diffusion Model
AI	Artificial Intelligence	LSTM	Long Short-Term Memory
ANSP	Air Navigation Service Provider	LSZH	Zurich Airport
APM	Aircraft Performance Model	MAE	Mean Absolute Error
ARPM	Airline Procedure Model	ML	Machine Learning
ATC	Air Traffic Control	MSE	Mean Squared Error
ATM	Air Traffic Management	PCA	Principal Component Analysis
BADA	Base of Aircraft Data	R14	Runway 14
CNN	Convolutional Neural Network	ResNet	Residual Network
CSDI	Conditional Score-based Diffusion Imputation	RNN	Recurrent Neural Network
CVAE	Conditional Variational Autoencoder	SDE	Stochastic Differential Equation
DDPM	Denoising Diffusion Probabilistic Model	SDG	Synthetic Data Generation
DGM	Deep Generative Model	SGD	Stochastic Gradient Descent
DM	Diffusion Model	STAR	Standard Terminal Arrival Route
ELBO	Evidence Lower Bound	t-SNE	t-Distributed Stochastic Neighbor Embedding
FiLM	Feature-wise Linear Modulation	TCVAE	Temporal Convolutional Variational Autoencoder
GAN	Generative Adversarial Networks	TMA	Terminal Maneuvering Area
GDPR	General Data Protection Regulation	TRTR	Train on Real, Test on Real
KL	Kullback–Leibler	TSTR	Train on Synthetic, Test on Real
		VAE	Variational Autoencoder

List of Figures

2.1	t-SNE overlays (real in red, generated in blue) across Dublin data fractions using a deep generative model, demonstrating good generative diversity with just 5% of the training data [3].	5
2.2	The recurrent architecture of a Recurrent Neural Network (RNN), illustrating how the hidden state is passed across sequential timesteps [21].	6
2.3	The internal architecture of an Long Short-Term Memory (LSTM) cell, showcasing the cell state (top horizontal line) and the gating mechanisms (bottom pathways) that regulate information flow [21].	6
2.4	Comparison of information routing in RNNs and Self-Attention mechanisms. Self-Attention allows global receptive fields in a single layer [21].	7
2.5	The fundamental cross-correlation (convolution) operation of a Convolutional Neural Network (CNN), where a learnable kernel slides across an input to produce a feature map [21].	8
2.6	Organization of the BADA model [24].	10
2.7	Example trajectory generated by BADA (ROC over time) [24].	10
2.8	Example trajectory generated by BADA (Airspeed over time) [24].	10
2.9	Organization of the OpenAP model [25].	11
2.10	Example of a descent trajectory generated by OpenAP in terms of altitude, airspeed and vertical rate with random variation (gray) around the characteristic (red) descent profile [25].	11
2.11	Error of extrapolated synthetic flights in different countries compared to EUROCONTROL flight plan data in 2015 [29].	12
2.12	Real aircraft trajectories on a straight-line flight plan, demonstrating positional deviation [30].	12
2.13	Taxonomy of deep generative modeling approaches based on density function estimation. Adapted from [33].	13
2.14	Architectural schema of a Generative Adversarial Network [36].	14
2.15	Architectural schema of a Variational Autoencoder [36].	15
2.16	Architectural schema of a Diffusion Model [36].	16
4.1	Methodological workflow outlining the progression from raw data extraction to synthetic trajectory evaluation.	22
4.2	Visual representation of three of the raw extracted trajectories from the OpenSky Network.	23
4.3	Coordinate based representation of the raw extracted trajectories from the OpenSky Network over the area of interest.	24
4.4	Comparison of spatial threshold filtering. Left: Clustering results with a relaxed spatial threshold of 75,000 m. Right: Refined clustering results with a strict spatial threshold of 5,000 m, isolating the core approach bundles and discarding outliers.	25
4.5	Example of a 1D UNet architecture with 4 compressing steps. Note that the feature channel is doubled while the feature size is halved after passing one layer of the encoder. The decoder mirrors this process, halving the feature channel and doubling the feature size after each layer. The blue arrows represent the skip connections, transferring the local details. Based on [46].	26

4.6	Architecture diagram demonstrating the injection of Time and Conditional embeddings into the 1D U-Net via Feature-wise Linear Modulation (FiLM). The unified global embedding vector modulates the spatial features across every hierarchical depth level of the denoiser.	27
4.7	Isolated Cluster 1 (from Figure 4.4) after applying the 5,000 m spatial threshold.	30
4.8	Generated coordinate trajectories (N = 30).	30
4.9	Expanded generation of synthetic coordinate trajectories (N = 1000 samples).	30
4.10	Raw synthetic trajectories generated by the model.	30
4.11	The real empirical coordinate based trajectories from the three main spatial clusters shown together.	32
4.12	Self-diagnostic PCA coverage results, validating metric sensitivity. The perfect baseline showcases complete distributional overlap, while the corrupted baseline exhibits clear separation.	35
4.13	Self-diagnostic 1-NN Distance Analysis for memorization checks. Note the expected shifts in the intra-set and inter-set nearest neighbor distances when trajectories are fundamentally disjoint.	35
5.1	1000 synthetic trajectories for Cluster 1 after post-processing filtering, successfully removing the artificial switchback artifacts.	38
5.2	Corresponding coordinate based generated synthetic trajectories matching the three main clusters (N = 1000 samples).	39
5.3	Representation of three random generated synthetic trajectories (kinematics).	39
5.4	Visual diversity comparison using PCA.	40
5.5	Visual diversity comparison using t-SNE.	40
5.6	Visual diversity plots evaluating the distribution of synthetic versus real trajectories.	40
5.7	Trajectory memorization check analyzing the 1-NN distances to verify the absence of exact training copies.	41

List of Tables

- 4.1 Self-diagnostic pipeline results validating the sensitivity of the evaluation metrics. The Perfect Baseline evaluates the real dataset against itself, while the Degraded Baseline evaluates it against a systematically corrupted subset. 34
- 5.1 Summary of trajectory evaluation results. 41
- 5.2 Comparison of aircraft type distribution between real and synthetic trajectories. The ten most frequent types are shown individually, while the remainder are aggregated. 42
- 5.3 Performance comparison of the proposed 1D Diffusion Model (DM) against the reference TimeGAN model. 43
- B.1 Hardware specifications and training time benchmarking. 52
- B.2 Sampling time benchmarking for different batch sizes and denoising steps. 52

Introduction

1.1. Motivation

As Air Traffic Management (ATM) systems face escalating demands and increasing complexity, advancing automation by leveraging Artificial Intelligence (AI) and Machine Learning (ML) has become crucial for maintaining both safety and operational efficiency [1, 2]. However, the effectiveness of these advanced ML models is inherently constrained by the availability of comprehensive, high-quality datasets required for training. In practice, acquiring sufficient trajectory data is frequently hampered by high costs, logistical challenges, and privacy regulations, a limitation widely recognized as the “data access problem” [2].

Furthermore, exploring rare events, edge cases, and “what-if” scenarios (such as go-arounds or holding patterns) requires datasets that are far more varied and extensive than what is typically available from real-world historical records [3]. To overcome these severe data scarcity issues, Synthetic Data Generation (SDG) has emerged as a highly promising solution. By synthesizing artificial yet realistic flight trajectories, researchers can effectively bridge the data gap, augmenting ML models and conducting large-scale safety analyses without disclosing sensitive operational information [1, 4].

1.2. Problem Statement

While synthetic trajectory generation holds immense potential, generating complex, flyable terminal trajectories remains a significant technical challenge [5]. Existing approaches to trajectory generation are generally divided into two categories: model-driven and data-driven methods. Model-driven methods rely on established flight dynamics and kinematic equations, which guarantee that the generated paths adhere to the laws of aerodynamics and physical performance boundaries. However, because of their deterministic nature, these models struggle to capture the full variability of real-world air traffic operations, such as sudden maneuvers commanded by Air Traffic Control (ATC) interventions, human factors, or dynamic weather uncertainties [6, 7, 8].

Conversely, data-driven generative models—such as Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), and Diffusion Models (DMs)—excel at learning complex statistical distributions directly from historical data, naturally capturing operational uncertainties [5, 3]. However, existing diffusion-based approaches to trajectory synthesis often rely on heavy architectures borrowed from other domains. For instance, discrete Transformer architectures (like Conditional Score-based Diffusion Imputation (CSDI)) or over-parameterized 2D U-Nets adapted from image generation are computationally expensive and not natively suited for the continuous, high-frequency nature of kinematic time-series data.

The difficulty is particularly pronounced in Terminal Maneuvering Areas (TMAs), where flights exhibit sudden changes in heading and altitude due to high traffic density and complex approach procedures. Generating complete 4-dimensional terminal trajectories requires a model that can capture long-range dependencies across both departure and arrival phases whilst simultaneously reproducing fine-grained local variations. Standard image-based or discrete models often rely on computationally heavy embeddings to

maintain this local temporal continuity, highlighting the need for a streamlined, domain-specific architecture tailored explicitly for continuous 1D kinematic sequences.

1.3. Research Scope and Objective

The primary objective of this thesis is to leverage novel deep generative modeling techniques to develop a highly stable conditioned 1D U-Net DM framework capable of synthesizing realistic 4D aircraft landing trajectories inside the TMA. Crucially, the model aims to act as a robust generative foundation operating directly within a Kinematic State Space. By learning the underlying kinematic principles of flight (how to fly) rather than memorizing spatial waypoints (where to fly), this representation aims to mitigate geographic leakage, unlocking the ability to generate valid trajectories across unobserved terminal areas [5, 3].

By utilizing a score-based generative approach driven by a forward and reverse diffusion process, the proposed 1D architecture offers a streamlined, computationally efficient solution to the generative learning trilemma (balancing high-quality sampling, mode coverage, and fast inference) [9], providing a stable, high-quality generation of continuous time series trajectories that overcomes the mode collapse issues prevalent in other GAN methodologies [2]. Ultimately, the synthetic samples generated by the DM will be rigorously evaluated against objective criteria using a multi-faceted statistical evaluation pipeline to verify their temporal fidelity, similarity, overfitting check, and downstream usefulness. The formal definition of the specific research questions designed to evaluate this objective is detailed later in the Research Proposal (Chapter 3).

1.4. Thesis Outline

To comprehensively address the problem statement and research questions, the remainder of this thesis is structured as follows:

Part I: Literature Study & Research Definition This part provides an extensive review of the state-of-the-art in aircraft trajectory generation, examining the trade-offs between model-driven flight dynamics and data-driven methods, and formalizes the research proposal.

Part II: Methodology This part details the proposed research methodology, beginning with data collection, preprocessing, and coordinate transformation. It formally introduces the generative model architectures selected for this study, describing the neural network design, the conditional inputs, and the diffusion training pipeline.

Part III: Results & Discussion This part presents the quantitative outcomes of the model training and generation process. The synthesized trajectories are evaluated against real historical data using specific distance metrics to assess distributional alignment and temporal consistency. Crucially, it includes a comprehensive verification pipeline, a comparison against reference baselines, and a detailed discussion of the findings.

Part IV: Closure The final part synthesizes the research findings, directly answering the proposed research questions. It discusses the conclusions of the study and outlines potential avenues for future work, including the possible integration of physics-informed models and real-time collision risk evaluation systems.

Part I

Literature Study & Research Definition

2

Literature Review and Theoretical Background

The development of synthetic aircraft trajectory generation models is a multifaceted challenge that integrates aeronautical engineering and deep generative modeling. This chapter establishes the theoretical framework required for this thesis. Section 2.1 provides an overview of synthetic data and the data access problem. Section 2.2 details the mathematical representation of 4D aircraft trajectories, contrasting spatial and kinematic state spaces. Existing open data sources are reviewed in Section 2.4. The taxonomy of generative modeling techniques is then laid out: traditional methods are covered in Section 2.5, while modern deep generative models—including VAEs, GANs, and Diffusion Models—are introduced in Section 2.6. Finally, the framework for evaluating synthetic trajectories is outlined in Section 2.7.

2.1. Synthetic Data: Definition and Overview

Synthetic data can be defined as artificial information generated by an AI algorithm that has been trained on a real dataset [10]. While this data is designed to be statistically representative of historic, real-world data, it is purely artificial and does not correspond to any existing objects or events [11]. Within machine learning pipelines, obtaining high-quality training volumes often poses significant challenges, collectively known as the *data access problem* [12, 11]. This problem is broadly categorized into three pillars:

1. **Data Quality:** Low-quality data can lead to erroneous model understanding, false predictions, and diminished usefulness [13, 14].
2. **Data Scarcity:** The sheer unavailability of sufficient data volumes heavily limits the complexity of patterns a model can capture [15].
3. **Data Privacy and Fairness:** Sensitive information cannot be freely shared or utilized due to strict privacy regulations, such as the General Data Protection Regulation (General Data Protection Regulation (GDPR)) [16, 17].

Synthetic data provides a robust methodology to circumvent these challenges, offering high-quality, privacy-compliant datasets that unlock the potential of emerging machine learning models.

Different forms of synthetic data are demanded to serve the diverse data requirements of various industries. Synthetic tabular, image, text, audio, spatial, and video data have found immense success across fields ranging from autonomous driving to natural language processing. However, due to the inherent temporal dependencies and sequential nature of aircraft kinematics, the primary focus of this thesis is specifically on **Time Series** synthetic data. Time series generation has gained significant traction across industries, such as enhancing predictive tasks in financial trading strategies [18], simulating electricity demand [19], and modelling weather patterns [20]. By conceptualizing an aircraft trajectory as a continuous, multivariate time series, deep generative models can be leveraged to mathematically reconstruct the complex temporal dynamics of flight.

2.2. Representation of 4D Aircraft Trajectories

In the context of ATM, Olive et al. [4] describe aircraft trajectories as a set of timestamped coordinates that must reflect both the physical behavior of the aircraft and the statistical properties of the airspace. Normally represented by the four-dimensional tuple (x, y, z, t) [2, 4], where x , y and z are the spatial coordinates (latitude, longitude and altitude), and t is the timestamp. These trajectories are inherently variable in length due to differences in flight duration, aircraft type, and operational procedures.

However, recent research by Larsen et al. [3] demonstrates that training generative models on purely kinematic representations rather than raw spatial coordinates prevents “geographic leakage”(where a model learns *where to fly* rather than *how to fly*), allowing the network to learn universal flight dynamics and become truly airport-agnostic. As shown in Figure 2.1, their findings prove that models trained on kinematic state-spaces can efficiently transfer knowledge from data-rich hubs to data-scarce target airports.

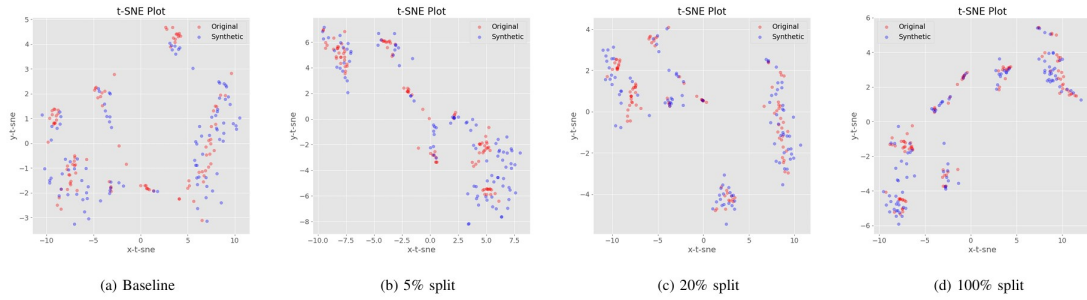


Figure 2.1: t-SNE overlays (real in red, generated in blue) across Dublin data fractions using a deep generative model, demonstrating good generative diversity with just 5% of the training data [3].

To be able to leverage these generalization capabilities, this thesis actively adopts their methodology. Consequently, the synthetic trajectories ($\mathbf{x}_t$) generated in this research are parameterized purely within the kinematic state space, utilizing track angle (ψ_t), groundspeed ($v_{g,t}$), altitude (h_t), and elapsed time (t):

$$\mathbf{x}_t = [\psi, v_{g,t}, h_t, t] \quad (2.1)$$

Once the optimal state-space variables are defined, these highly variable sequences must be standardized for deep learning architectures through temporal alignment. A proven methodology involves resampling the flight kinematics to generate trajectory vectors with a fixed length L of tracking observations (where L must be large enough to maintain fine-grained trajectory representation). This is achieved by linearly interpolating the states for timestamps $t = \{0, \tau, 2\tau, \dots, T\}$ evenly spaced throughout the total flight duration T , where $\tau = \frac{T}{L-1}$ [4].

2.3. Temporal Dynamics and Sequence Generation

As it has been mentioned, unlike static imagery, where features exhibit purely spatial correlations, flight trajectories are multivariate time series characterized by strict sequential dependencies and physical causality. Generating such trajectories requires architectures capable of modeling stepwise temporal dynamics.

2.3.1. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks were developed to process data sequentially. As explained comprehensively by Zhang et al. [21], an RNN passes a hidden state vector forward through time, allowing the network to retain a “memory” of the sequence up to the current timestep (see Figure 2.2).

However, standard RNNs notoriously suffer from the vanishing gradient problem, making it nearly impossible for them to learn long-term dependencies across lengthy sequences like full flight paths [21]. This problem occurs during backpropagation through time, where error gradients must be repeatedly multiplied by the network's weight matrices for each historical timestep. If these weight values are small (less than one), the gradients exponentially decay—or “vanish”—to zero as they propagate backwards. Consequently, the network fails to pass any meaningful learning signal to earlier timesteps, effectively causing the model to “forget” the beginning of the trajectory by the time it reaches the end.

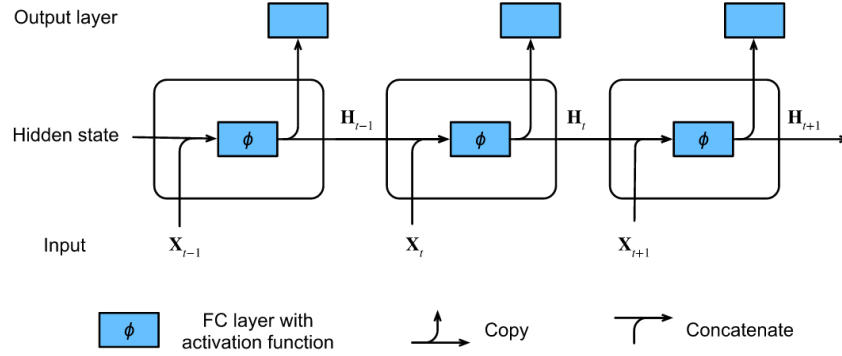


Figure 2.2: The recurrent architecture of a RNN, illustrating how the hidden state is passed across sequential timesteps [21].

2.3.2. Long Short-Term Memory (LSTM)

To resolve this limitation, Long Short-Term Memory networks were introduced. As detailed by Zhang et al. [21], LSTMs are a specialized type of RNN engineered explicitly to remember information over extended time horizons. They achieve this through a sophisticated internal architecture comprising a continuous “cell state” and three regulating mechanisms known as gates (illustrated in Figure 2.3). The cell state acts as a conveyor belt, running straight down the entire sequence with only minor linear interactions, allowing crucial information to flow forward uninterrupted.

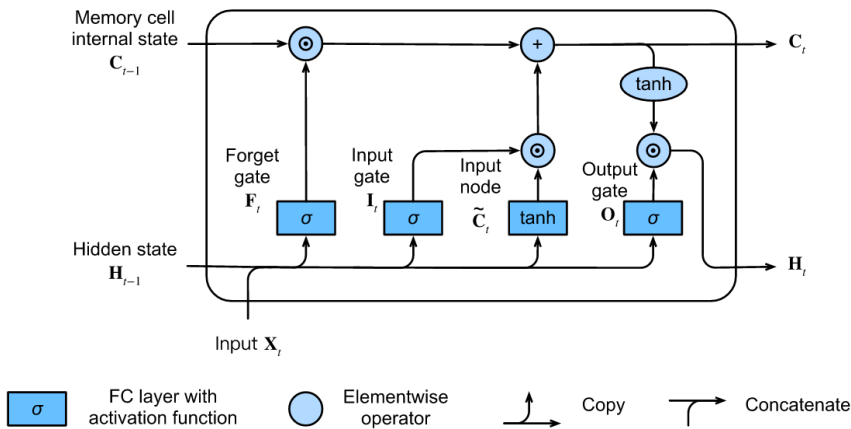


Figure 2.3: The internal architecture of an LSTM cell, showcasing the cell state (top horizontal line) and the gating mechanisms (bottom pathways) that regulate information flow [21].

The flow of information into and out of the cell state is rigorously controlled by three distinct layers:

- **Forget Gate:** Evaluates the current input and the previous hidden state to decide what historical information is no longer relevant and should be discarded from the cell state.

- **Input Gate:** Determines what new, incoming information from the current timestep is critical and should be written into the cell state.
- **Output Gate:** Decides what specific parts of the updated cell state should be output as the new hidden state, effectively determining the immediate action or prediction for the current timestep.

In the context of trajectory modeling, this gating mechanism is paramount. It allows the neural network to remember long-term strategic dependencies—such as an aircraft’s intention to land on a specific runway or its entry into a holding pattern—while simultaneously possessing the ability to filter out short-term, high-frequency noise. Consequently, LSTMs remain a foundational component in modern time-series analysis and sequential forecasting tasks. However, despite effectively mitigating the vanishing gradient problem, LSTMs still process sequences autoregressively—one step at a time. This inherent sequential bottleneck prevents computational parallelization during training and still causes the model to struggle when capturing dependencies across extremely long temporal sequences.

2.3.3. Self-Attention and Transformers

To overcome the sequential processing bottlenecks and long-range dependency limitations of recurrent architectures like LSTMs, the Self-Attention mechanism was introduced [21]. Unlike RNNs, which compress past information into a single hidden state one step at a time, Self-Attention computes a weighted representation by allowing every timestep to directly attend to every other timestep in the sequence simultaneously, regardless of their physical distance in time (see Figure 2.4).

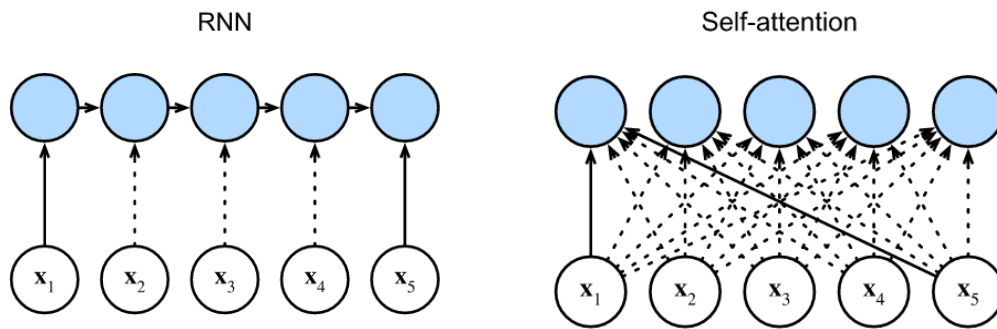


Figure 2.4: Comparison of information routing in RNNs and Self-Attention mechanisms. Self-Attention allows global receptive fields in a single layer [21].

This paradigm shift forms the foundation of the Transformer architecture, enabling vastly superior long-range dependency modeling. However, this architecture introduces a fundamental computational trade-off. Because Self-Attention computes weights for every possible pair of timesteps, its computational complexity grows quadratically with sequence length ($\mathcal{O}(N^2)$), compared to the linear complexity ($\mathcal{O}(N)$) of RNNs. While this makes Self-Attention exceptionally memory-intensive for extremely long sequences, the absence of a sequential bottleneck allows the matrix multiplications to be heavily parallelized across modern hardware. In practice, for short to medium sequences, this parallelization completely offsets the quadratic mathematical cost, resulting in faster training times than sequential LSTMs.

Consequently, in modern generative modeling—including state-of-the-art Diffusion Models—attention mechanisms are heavily utilized to capture complex spatial and temporal correlations simultaneously, empowering the model to generate robust sequences.

2.3.4. Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) have long been the dominant architecture for spatial data such as 2-dimensional images [21]. The core mechanism of a CNN is the cross-correlation operation (commonly

referred to as convolution), where a small, learnable window (the kernel) slides across the input to extract localized features (see Figure 2.5). Because the same kernel weights are reused across the entire input, CNNs drastically reduce the number of parameters compared to fully connected networks and naturally achieve translation invariance (the ability to recognize a feature regardless of where it appears in the input).

Input		Kernel		Output																	
<table border="1" style="border-collapse: collapse;"> <tr><td>0</td><td>1</td><td>2</td></tr> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>6</td><td>7</td><td>8</td></tr> </table>	0	1	2	3	4	5	6	7	8	*	<table border="1" style="border-collapse: collapse;"> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	=	<table border="1" style="border-collapse: collapse;"> <tr><td>19</td><td>25</td></tr> <tr><td>37</td><td>43</td></tr> </table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Figure 2.5: The fundamental cross-correlation (convolution) operation of a CNN, where a learnable kernel slides across an input to produce a feature map [21].

While natively designed for 2D spatial grids (like pixels in an image), CNNs can be highly effective for 1D time-series analysis by treating the temporal axis as a spatial dimension. This effectiveness stems from a fundamental inductive bias: the kernel design inherently assumes that data points close to each other in time are highly related, making CNNs naturally suited to temporal dependencies.

Although individual convolution operations are local (don't capture large scale), advanced CNN architectures (such as the U-Net) overcome this by employing deep, hierarchical structures that progressively compress (downsample) the sequence. By sequentially compressing the data, each subsequent convolutional layer effectively analyzes a much wider slice of the original timeline. This elegant expansion of the network's receptive field allows it to efficiently capture both high-frequency local variations and long-range global dependencies, completely bypassing the computational sequential bottlenecks inherent to RNNs or LSTMs, and without incurring the massive quadratic memory costs of full Self-Attention.

Crucially, because the DM framework originated in image synthesis, standard implementations overwhelmingly rely on 2D CNNs. However, when adapting these models to 1D kinematic sequences (dependent only on time), naively relying on 2D convolutions by treating independent state variables as pseudo-spatial dimensions forces the network to perform unnecessary, computationally expensive spatial operations. This introduces massive over-parameterization and results in a severe waste of computational resources, creating a critical need for specialized 1D CNN architectures natively tailored to the true dimensionality of trajectory time-series.

2.4. Available Open Data Sources

To train generative architectures or constrain them with physical boundaries, distinct classes of aviation data are required. These open data sources can be broadly categorized into two types: surveillance databases providing historical trajectories, and performance models providing aerodynamic specifications.

2.4.1. Category 1: Surveillance and Trajectory Databases

These databases provide the raw historical flight data from which 4D trajectories are reconstructed. The **EUROCONTROL R&D Archive** provides historical filed flight plans and actual flown routes for commercial flights over Europe [22]. The **OpenSky Network** is an open-access surveillance network that collects live Automatic Dependent Surveillance-Broadcast (ADS-B) and Mode S messages [23]. OpenSky provides high-frequency, raw state vectors encompassing aircraft position, velocity, and status variables. Because the core objective of this thesis is to accurately model the temporal dependencies of aircraft *kine-*

matics (groundspeed, altitude, track, and time), the high-resolution kinematic state vectors provided by the **OpenSky Network** will serve as the primary data source for the generative models developed herein.

2.4.2. Category 2: Aircraft Performance Models (APMs)

Unlike surveillance databases, APMs do not contain historical flight paths. Instead, they catalogue the physical limitations and aerodynamic coefficients of specific aircraft types. **BADA (Base of Aircraft Data)** is the industry-standard repository providing detailed parameters governing aircraft aerodynamics, thrust limits, and weight characteristics across various flight phases [24]. **OpenAP** serves as an open-source alternative, deriving performance models from crowdsourced flight data [25]. Consequently, these APMs can be used not only to generate model-driven trajectories but also to enforce physics-informed constraints on deep generative models.

2.5. Traditional Trajectory Generation Methods

Before the advent of deep learning architectures, trajectory generation was heavily reliant on model-driven and macroscopic stochastic methodologies. These classical approaches encompass deterministic mathematical models as well as statistical extrapolation, serving as the foundational baseline against which modern generative AI is evaluated.

2.5.1. Model-Driven Rationale and Limitations

Model-driven aircraft trajectory generation relies on mathematical kinematic and aerodynamic models to calculate trajectories based on environmental constraints, optimizing objectives like fuel consumption and flight time [4]. Model-driven generation is considered the classical approach and comprises most existing methods in literature [1]. The optimization process can take different forms; for example, the use of dynamic programming to generate optimal trajectories given designated arrival slots [26] or adding a wide variety of constraints such as avoiding restricted airspace [27] and adverse weather scenarios [28].

Physical and statistical modeling of aircraft performance characteristics serves as an alternative foundation for generation. BADA and OpenAP are industry-wide recognized examples of such models, each capable of deterministically generating flyable trajectories based on predefined aerodynamic constraints.

While capable of generating highly realistic and strictly flyable paths, the fundamentally deterministic nature of these models poses a critical limitation: they struggle to encapsulate the true statistical uncertainty and stochastic variations inherent in real-world radar observations, translating into an inability to generate larger sets of diversified trajectories. Some attempts to fix this have been made, including slight variations in the models, however, this included randomness might not be representative of what is observed [1, 4].

2.5.2. BADA

BADA is a comprehensive repository developed by EUROCONTROL that catalogues parameters governing aircraft aerodynamics, propulsion, weight, and flap configurations across various operating conditions [24]. It generates trajectories using a piece-wise approach structured around the APM and the Airline Procedure Model (ARPM) (see Figure 2.6). The APM treats the aircraft as a point mass, computing aerodynamic, propulsive, and gravitational forces via an action model. A subsequent motion model utilizes these kinematic relations to calculate the physical performance of the aircraft along its trajectory, while a limitations model enforces strict adherence to the safe flight envelope. Different operational possibilities are handled by the operations model and the ARPM contains information on nominal aircraft procedures, that vary for airspace. Figures 2.7 and 2.8 illustrate some kinematic variables of an example trajectory generated by the BADA model.

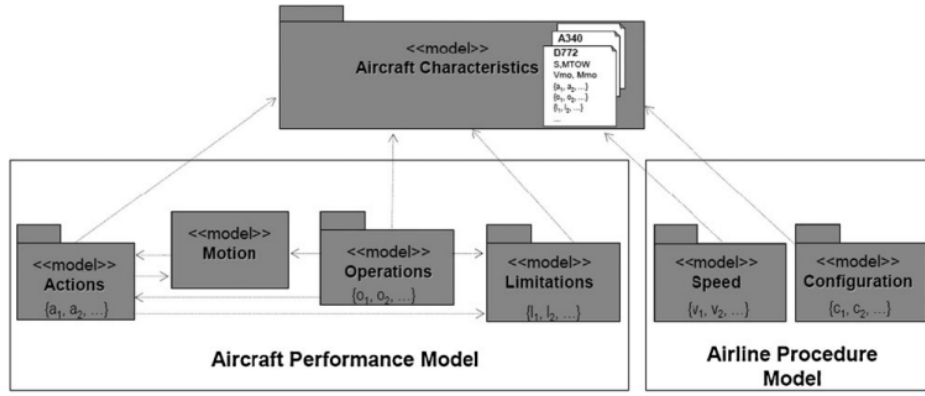


Figure 2.6: Organization of the BADA model [24].

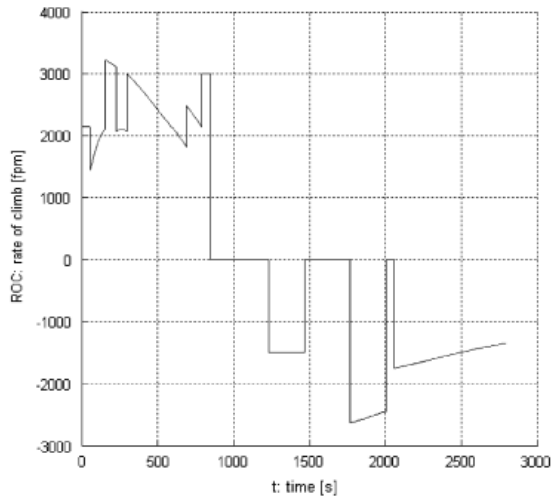


Figure 2.7: Example trajectory generated by BADA (ROC over time) [24].

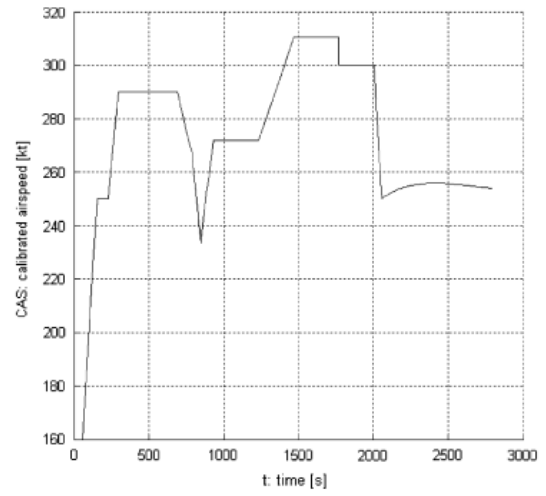


Figure 2.8: Example trajectory generated by BADA (Airspeed over time) [24].

2.5.3. OpenAP

OpenAP serves as a prominent open-source alternative to BADA, deriving its performance models from crowdsourced ADS-B flight data [25]. The OpenAP architecture (illustrated in Figure 2.9) is divided into four primary modules: aircraft properties (engine configurations and performance limits), kinematics (motion equations spanning seven distinct flight phases), dynamics (which extends the kinematics by computing point-mass forces), and utilities (navigation data and conversions). Similar to BADA, OpenAP includes a dedicated generator capable of concatenating individual flight segments using predefined altitude and Mach number guides to construct full synthetic trajectories (see Figure 2.10 for an example).

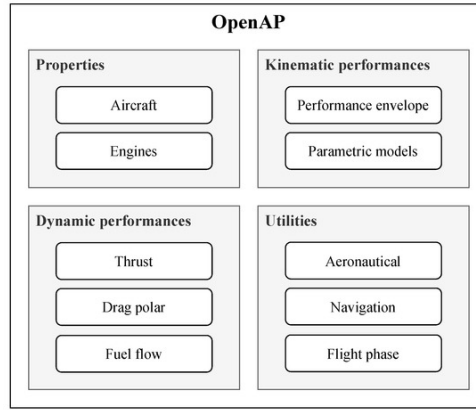


Figure 2.9: Organization of the OpenAP model [25].

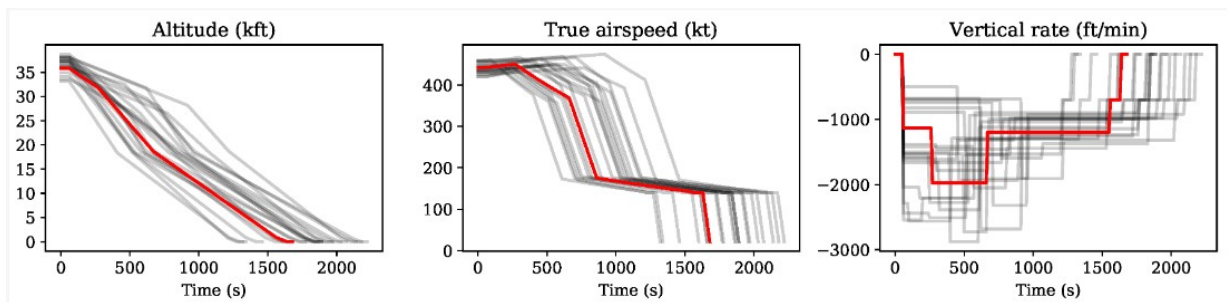


Figure 2.10: Example of a descent trajectory generated by OpenAP in terms of altitude, airspeed and vertical rate with random variation (gray) around the characteristic (red) descent profile [25].

2.5.4. Flight Plan Statistical Extrapolation

While the previously mentioned APMs are driven by physical equations, an alternative macroscopic approach to generating entire synthetic traffic scenarios relies on the statistical extrapolation of verified flight plan databases. Thomas Gräupl [29] used reference days from databases (such as those compiled by EUROCONTROL) to compute hourly aircraft generation rates for departure and arrival pairs. These rates were then extrapolated to future dates randomly distributing the average values within a $\pm 10\%$.

While useful for simulating large-scale traffic density and network load, this methodology simply concatenates rigid flight segments (e.g., constant climb, cruise, and descent) and introduces rudimentary uniform variance. It does not authentically recreate the complex temporal dynamics of individual flight kinematics. Furthermore, validation studies have demonstrated significant spatial errors in the resulting simulations; for example, the extrapolated models often exhibit unrealistic trajectory concentrations at major airports when compared against true radar-correlated data (see Figure 2.11) [29].

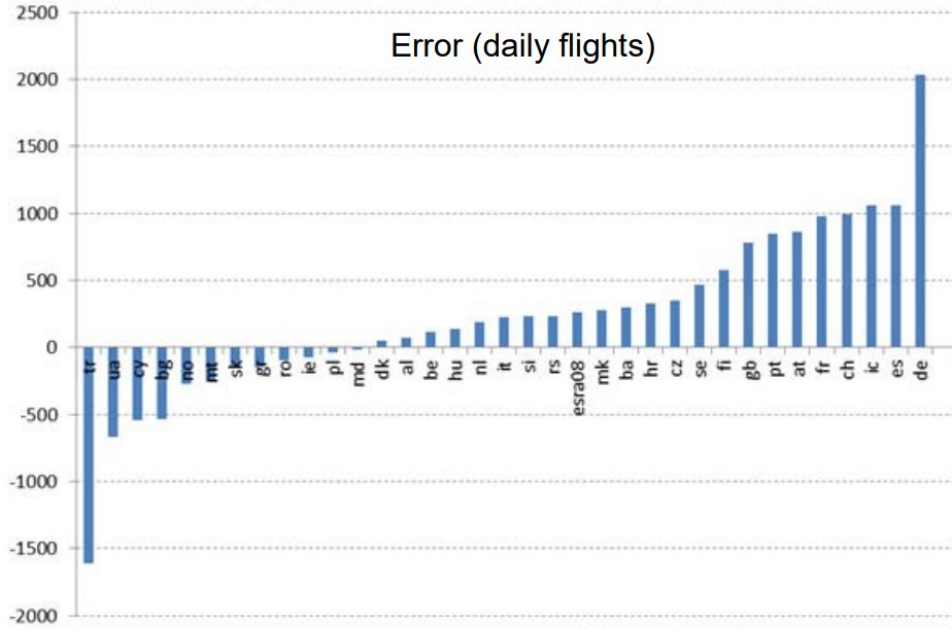


Figure 2.11: Error of extrapolated synthetic flights in different countries compared to EUROCONTROL flight plan data in 2015 [29].

2.5.5. Stochastic Macroscopic Models (Markov Chains)

In reality, even when an aircraft is instructed to fly a straight line between two waypoints, the true trajectory exhibits continuous, random deviations due to wind, navigation inaccuracies, and control variations (as illustrated in Figure 2.12).

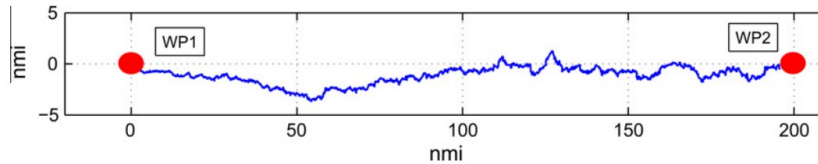


Figure 2.12: Real aircraft trajectories on a straight-line flight plan, demonstrating positional deviation [30].

Observations presented by Paielli and Field demonstrate that aircraft position error for a given time is well-represented by a centered Gaussian density, the standard deviation of which increases with time [31]. Consequently, framing this uncertainty as a stochastic process is a highly valid model for characterizing aircraft trajectories. Under this assumption, the evolution of the aircraft's position can be mathematically formalized via the following Stochastic Differential Equation (SDE) [30]:

$$d\mathbf{X}_t = \mathbf{v}dt + \sigma_t d\mathbf{W}_t \quad (2.2)$$

where $\mathbf{X}_t$ is the position of the aircraft at time t , $\mathbf{v}$ is a three-dimensional speed vector, σ_t represents a squared correlation matrix, and $\mathbf{W}_t$ is a standard three-dimensional Brownian motion (continuous-time random fluctuations) representing the random deviation [32].

While this mathematically formalized stochasticity is extremely valuable for applications such as conflict probability estimation in uncontrolled airspace [30], traditional macroscopic models are severely restricted by their rigid physical assumptions. Because they typically rely on fixed transition matrices to

simulate simple 2-dimensional flight segments with constant speed and altitude, they lack the capacity to generate the complex, high-frequency kinematic variations inherent to realistic terminal maneuvers [2]. However, when the foundational mathematics of a Markov chain—specifically the ability to formulate probabilistic, step-by-step state transitions—is decoupled from these rigid physical equations and integrated with deep neural networks, it becomes exceptionally powerful. In fact, this exact mathematical principle serves as the theoretical backbone for modern data-driven Diffusion Models, which rely entirely on Markov chains to govern both their forward noise-injection process and their parameterized reverse denoising process.

2.6. Data-Driven Methodologies: Deep Generative Models

Recent literature has transitioned from traditional physics-based simulation to data-driven Deep Generative Models to capture the complex, stochastic nature of flight operations. Generative models aim to learn the underlying probability distribution ($p(x)$) of historical flight data, enabling the sampling and synthesis of entirely new, physically plausible trajectories. Figure 2.13 shows how these architectures can generally be categorized by how they estimate this density function: whether explicitly, by defining a tractable or approximate probability density function, or implicitly, by learning a direct mapping from random noise to the data space without modeling the underlying density [33].

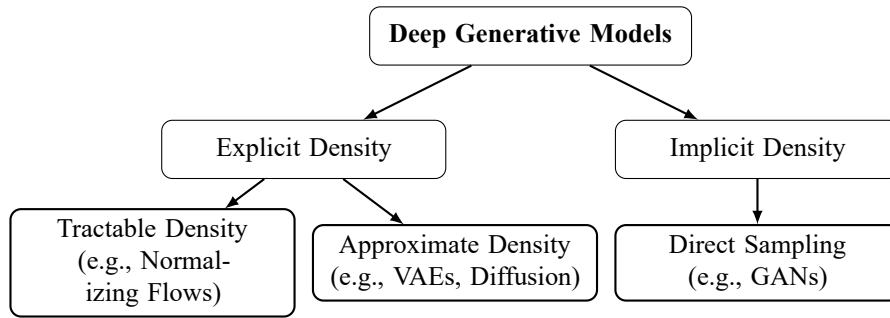


Figure 2.13: Taxonomy of deep generative modeling approaches based on density function estimation. Adapted from [33].

2.6.1. Maximum Likelihood Estimation

To simplify the discussion and establish a common mathematical foundation, it is highly beneficial to focus on generative models that operate via the principle of maximum likelihood (probability) estimation. While some architectures, such as GANs, do not natively utilize maximum likelihood by default, they can be adapted to do so. By focusing on the maximum likelihood framework, it becomes easier to understand the core objective driving these models without being distracted by architectural nuances [33].

The fundamental principle of maximum likelihood is to define a parameterized model, $p_{\text{model}}(x; \theta)$, that provides an estimate of the true data distribution. The goal is to find the optimal parameters θ^* that maximize the likelihood the model assigns to the N trajectories in the training dataset:

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^N p_{\text{model}}(x^{(i)}; \theta) \quad (2.3)$$

In practice, this optimization is almost always performed in log-space. The logarithm transforms the product into a sum, which drastically simplifies the algebraic computation of derivatives and prevents numerical underflow when multiplying many extremely small probabilities together:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^N \log p_{\text{model}}(x^{(i)}; \theta) \quad (2.4)$$

Crucially, maximizing this log-likelihood is mathematically equivalent to minimizing the Kullback–Leibler (KL) divergence between the empirical distribution of the training data ($\hat{p}_{\text{data}}$ ¹) and the model's parameterized distribution (p_{model}):

$$\theta^* = \arg \min_{\theta} D_{\text{KL}}(\hat{p}_{\text{data}}(x) \parallel p_{\text{model}}(x; \theta)) \quad (2.5)$$

The KL divergence is a fundamental statistical measure that quantifies the difference, or information loss, between two probability distributions [34]. This equivalence highlights that explicitly optimizing $p(x)$ is simply the process of minimizing this distance, effectively forcing the model's synthesized distribution to perfectly overlap with the real-world historical flight distribution.

2.6.2. Generative Adversarial Networks (GANs)

Generative Adversarial Networks operate as implicit density models, meaning they never explicitly define or calculate the mathematical probability distribution $p(x)$ of the trajectory data. Instead, they learn to sample directly from the target distribution through an adversarial training paradigm, as illustrated in Figure 2.14. A GAN features two competing neural networks: a generator ($G(z)$) that transforms random noise into synthetic trajectories, and a discriminator ($D(x)$) tasked with distinguishing these fakes from real historical data [35].

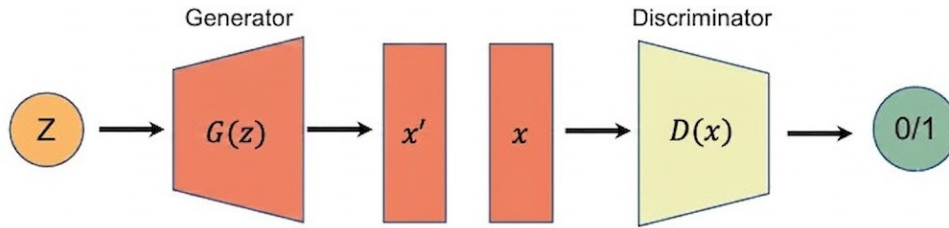


Figure 2.14: Architectural schema of a Generative Adversarial Network [36].

Through a two-player minimax game, the generator continuously learns to produce increasingly realistic samples to deceive the discriminator, while the discriminator learns to better distinguish real from synthetic data. The adversarial objective is formulated as:

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] \quad (2.6)$$

While GANs are highly proficient at capturing high-frequency variations and generating high-quality samples rapidly, they frequently suffer from what is known as *mode collapse*. Mode collapse occurs when the generator learns to produce only a narrow subset of realistic trajectories, entirely failing to capture the comprehensive diversity of the true historical distribution [2]. Despite this, Wijnands et al. explored the application of TimeGAN for generating landing trajectories, highlighting the necessity of strict data clustering to mitigate collapse, but still facing challenges.

2.6.3. Variational Autoencoders (VAEs)

Unlike GANs, Variational Autoencoders are explicit density models. They explicitly formulate a mathematical approximation of the data distribution $p(x)$ by optimizing a tractable lower bound to the true log-likelihood. This architecture comprises an encoder network ($q_{\phi}(z|x)$) and a decoder network ($p_{\theta}(x|z)$), as depicted in Figure 2.15. The encoder compresses high-dimensional input trajectories into a lower-

¹In practice, only a set of N samples from p_{data} is available, that is why $\hat{p}_{\text{data}}$ is used, representing the empirical distribution that places mass only on those N points, approximating p_{data}

dimensional continuous latent space Z , while the decoder attempts to reconstruct the original paths from this compressed representation [37].

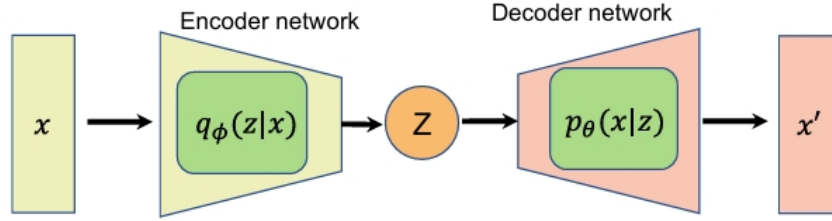


Figure 2.15: Architectural schema of a Variational Autoencoder [36].

The model is optimized by maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\theta, \phi}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z|x)) \quad (2.7)$$

This computable formulation explicitly balances reconstruction accuracy against a regularization term, ensuring the latent space closely approximates a predefined prior distribution $p(z)$. Furthermore, from a theoretical perspective, maximizing the ELBO with respect to the parameters θ and ϕ concurrently achieves two fundamental objectives:

1. It approximately maximizes the marginal log-likelihood $\log p_{\theta}(x)$, which directly improves the generative capability of the model to synthesize realistic trajectories.
2. It simultaneously minimizes the KL divergence between the approximate posterior $q_{\phi}(z|x)$ and the true posterior $p_{\theta}(z|x)$, thereby improving the accuracy of the encoder’s latent representations.

Within aviation, VAEs have been deployed to address stability concerns. Krauth et al. and Motte et al. utilized VAE architectures for trajectory synthesis [1, 38]. Krauth et al. demonstrated that using a *VampPrior* Temporal Convolutional Variational Autoencoder (TCVAE) allows the latent space to be structured into specific types of approaches or “pseudo-inputs”, enabling steerable generation. Motte et al. expanded this into a *Conditional VAE* (Conditional Variational Autoencoder (CVAE)) framework, injecting aircraft-type labels to ensure generated paths adhere strictly to the performance envelopes of diverse airframes.

Despite their mathematical elegance and stable training dynamics compared to GANs, standard VAEs suffer from inherent limitations in generative quality. While they optimize a variational lower bound, their architectural reliance on a restrictive, low-dimensional latent bottleneck, coupled with simple Gaussian priors and mean squared error reconstruction losses, typically results in blurry, over-smoothed samples. The mean squared error loss inherently penalizes variance, forcing the decoder to average over possible outcomes rather than picking sharp, distinct features. In the context of flight trajectories, this over-smoothing causes the model to lose critical high-frequency kinematics, washing out the sharp, precise maneuvers required for realistic flight paths.

2.6.4. Diffusion Models

To overcome the mode collapse of GANs, as well as the over-smoothed generative quality of VAEs, the field is currently converging towards Diffusion Models [3]. Like VAEs, Diffusion Models belong to the class of explicit density models because they directly estimate the gradient of the log-density of the data (the score function) and optimize a variational lower bound. They construct a sequence of invertible mathematical transformations (see Figure 2.16).

A forward diffusion process, $q(\mathbf{x}_t|\mathbf{x}_{t-1})$, incrementally destroys the structure of a pristine real trajectory ($\mathbf{x}_0$) over a fixed number of timesteps T_{diff} . At each step t , it systematically injects a small amount

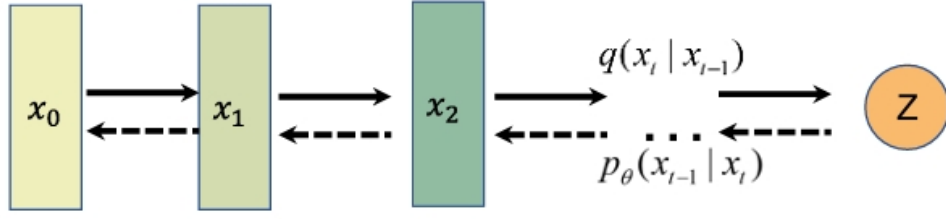


Figure 2.16: Architectural schema of a Diffusion Model [36].

of unparameterized Gaussian noise according to a variance schedule β_t [39]:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad (2.8)$$

where $\mathcal{N}$ denotes a Normal (Gaussian) probability distribution [40] parameterized by its mean (here, $\sqrt{1 - \beta_t} \mathbf{x}_{t-1}$) and its covariance matrix (here, $\beta_t \mathbf{I}$, with $\mathbf{I}$ representing the identity matrix). Over T_{diff} sequential steps, this formulation simulates a prolonged Brownian motion until the original data is entirely degraded into pure, structureless white noise ($\mathbf{x}_{T_{diff}}$).

Conversely, a parameterized reverse diffusion process, $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$, learns to systematically denoise this random distribution via a reverse Markov chain. Starting from pure noise $\mathbf{x}_{T_{diff}} \sim \mathcal{N}(0, \mathbf{I})$, the neural network executes exactly T_{diff} sequential denoising steps:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \quad (2.9)$$

Here, $\mu_\theta(\mathbf{x}_t, t)$ and $\Sigma_\theta(\mathbf{x}_t, t)$ represent the predicted mean and covariance of the reverse trajectory step, which are mathematically parameterized by the deep neural network's internal weights (θ). By repeatedly evaluating this parameterized distribution and subtracting a tiny fraction of noise at each step, the model iteratively reconstructs a highly diverse and realistic trajectory $\mathbf{x}_0$. While the mathematical foundation of this process relies on optimizing a complex variational lower bound over all T_{diff} timesteps, Ho et al. [39] demonstrated that this bound can be elegantly reduced to a highly intuitive surrogate objective; The neural network (ϵ_θ) is simply trained to minimize the Mean Squared Error (Mean Squared Error (MSE)) between the true noise ϵ injected during the forward process and the noise predicted by the network at step t :

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2] \quad (2.10)$$

This beautifully simple objective is what empowers Diffusion Models to achieve state-of-the-art sample quality without the instabilities of GANs. However, because standard Diffusion Models require these hundreds of sequential neural network passes (evaluating ϵ_θ at every step) to generate a single sample, they are notoriously slower at inference time compared to single-pass models like GANs or VAEs [41].

2.7. Evaluation of Synthetic Trajectories

Validating the quality of generated flight paths remains an open challenge in the aviation domain. Olive et al. [4] advocate for a holistic evaluation framework, arguing that trajectories must be assessed across two distinct dimensions: statistical similarity and operational flyability.

While statistical metrics (such as the energy distance or discriminative scores) are excellent for ensuring the generative model captures the full variance of the training data and avoids mode collapse, they are often blind to the laws of physics. A trajectory might appear statistically sound but require impossible acceleration rates or physically unfeasible turn angles. Consequently, operational checks—such as running the generated trajectories through ATM simulators like *BlueSky* to verify BADA aerodynamic constraints—are often required to guarantee that the synthetic data is practically viable for downstream applications.

3

Research Proposal

3.1. Introduction

As established in the preceding literature review, ATM heavily relies on accurate aircraft trajectory generation to evaluate operational safety—such as estimating conflict probabilities [30]—and to support airspace design. While data-driven Deep Generative Models such as GANs and VAEs excel at mimicking the statistical variability of human operations and ATC interventions [8, 2], they frequently suffer from significant limitations. Primarily, GANs frequently suffer from mode collapse and training instability, limiting their ability to reliably cover the full distribution of complex operational trajectories [2]. Furthermore, VAEs often suffer from over-smoothing and lack of high-frequency detail due to their restricted, low-dimensional latent bottlenecks. Moreover, it has been seen that models trained on spatial coordinates exhibit “geographic leakage,” effectively memorizing specific airport topologies, which severely limits their transferability to new or data-scarce airports [3].

This thesis proposes a novel framework to bridge these gaps by developing a highly efficient **Diffusion Model** operating in a Kinematic State Space, capable of high-fidelity in both large and small-scale scenarios and of highly diverse 4D terminal trajectory generation.

3.2. Identified Research Gaps

Despite rapid advancements in generative trajectory modeling, three critical research gaps persist in the literature:

- **Lack of Domain-Specific Architectures:** While diffusion models have shown immense promise—with recent work by Larsen et al. [3] concluding that direct Diffusion Models don’t suffer from mode collapse and outperform their Latent counterparts for trajectory generation—the current state-of-the-art relies on adapting over-parameterized 2D Convolutional U-Nets borrowed directly from image generation. Consequently, there is a lack of architectures explicitly designed for the continuous, time dependant kinematic nature of aircraft flights. These existing adaptations introduce unnecessary computational overhead and are not natively tailored to model the direct temporal continuity of flight maneuvers efficiently.
- **Suboptimal Conditioning Mechanisms:** Existing DMimplementations [3] primarily rely on unconditioned generation or crude concatenation for injecting variables. Integrating advanced modulation techniques—such as Feature-wise Linear Modulation (FiLM) [42]—directly into a 1D U-Net architecture remains an unexplored methodology, yet it offers a far more precise and computationally efficient way to steer trajectory generation based on specific operational constraints like aircraft type.
- **High-Dimensional Temporal Dependencies:** Generating complete terminal trajectories (from takeoff to terminal boundary for departures, or en-route entry down to final approach for arrivals)

requires maintaining long-range spatiotemporal coherence. Many existing architectures struggle to balance local fine-grained maneuvers with global flight trends across long time horizons [1, 8].

3.3. Research Objective and Questions

The primary objective of this research is to formulate, implement, and validate a conditioned DM that accurately synthesizes flyable terminal 4D aircraft trajectories.

To fulfill this objective, the thesis will address the following specific research questions:

1. How effective are 1D U-Net DMs trained with kinematic relations (Track/Speed/Altitude) in generating realistic synthetic aircraft trajectories during the landing phase?
2. What quantitative metrics are required to systematically evaluate the quality of synthetic aircraft trajectories?
3. How does the performance of a 1D U-Net DM compare against traditional GAN models?

3.4. Proposed Methodology and Work Packages

To bridge the identified gaps and answer the proposed questions, this thesis proposes a streamlined, highly stable **Conditional 1D U-Net Diffusion Model**. The core diffusion process, noise scheduling framework, and FiLM-based conditioning mechanisms are actively leveraged from the foundational work by Sokolov et al. [43], who originally designed these techniques to generate vehicle velocity and acceleration profiles through a trajectory. This thesis substantially adapts their framework for complex aviation environments by transitioning to a multi-variate **Kinematic State Space**. Furthermore, rather than relying on the complex, discrete Transformer imputers utilized in their original codebase to handle time-series structures, this architecture directly injects a **global absolute temporal coordinate axis** into the data. By combining this explicit temporal awareness with a highly efficient 1D Convolutional U-Net, the model serves as a stable, **domain-specific** diffusion denoiser perfectly suited for generating continuous aircraft trajectories.

Much of the existing literature uses Zurich Airport (LSZH) as a case study for trajectory generation, with a particular focus on Runway 14 (R14). Aligning with this established approach, this thesis utilizes Zurich's R14 as the primary runway for model training and evaluation, leveraging its well-documented approach procedures and abundant historical data. Consequently, the scope of the model is restricted to the landing phase, which represents the most critical and well-documented phase for this runway.

The research execution is structured into five cohesive work packages, following established frameworks for synthetic trajectory generation projects:

Work Package 1: Data Collection and Preprocessing

Objective: To establish a high-quality dataset of realistic landing trajectories for model training and evaluation.

Tasks:

- Acquire kinematic historical flight data from the OpenSky Network for arrivals at LSZH runway 14.
- Filter and clean the data to remove anomalies, incomplete trajectories, non-landing operations and holding patterns.
- Resample the trajectories to a fixed sequence length of $L = 200$ observations using linear interpolation across the flight duration, ensuring the neural network processes uniform temporal vectors.
- Perform standard k-means clustering to identify distinct approach patterns and operational clusters, ensuring the model captures the full diversity of landing strategies.

Work Package 2: Establishing the Evaluation Framework

Objective: To develop a comprehensive suite of metrics capable of assessing the variety, temporal fidelity, downstream utility and overfitting check of the synthetic trajectories along with a self-diagnostic pipeline.

Tasks:

- Implement dimensionality reduction techniques (PCA and t-SNE) to evaluate trajectory diversity and avoid mode collapse.
- Train a LSTM Discriminator to assess dynamic realism (Discriminative Score).
- Implement Energy Distance metrics to quantify the multi-dimensional distribution similarity.
- Establish a Train on Synthetic, Test on Real (TSTR) framework using an LSTM Regressor to measure downstream utility.
- Confirm the absence of memorization (overfitting) by performing a 1-Nearest Neighbor (1-NN) distance analysis between synthetic and real trajectories.
- Construct a self-diagnostic pipeline utilizing perfect and degraded baselines to validate the evaluation metrics themselves.

Work Package 3: Model Development and Training

Objective: To construct and train the proposed diffusion architecture operating in the kinematic latent space.

Tasks:

- Develop the data processing pipeline to reliably project trajectories into the deterministic kinematic state space.
- Adapt the open-source codebase from Sokolov et al. [43] to implement the Conditional 1D U-Net, expanding their longitudinal vehicle framework into a multi-variate aircraft trajectory generator utilizing FiLM for condition injection.
- Formulate the noise-prediction objective function and define the discrete forward diffusion variance schedules (β_t).
- Train the full diffusion architecture on the preprocessed LSZH R14 dataset.

Work Package 4: Model Evaluation and Validation

Objective: To rigorously validate the performance of the trained DM and compare it against existing architectures.

Tasks:

- Generate a large synthetic dataset of landing trajectories using the trained DM in Work Package 3.
- Apply the evaluation framework established in Work Package 2 to compute all statistical and flyability metrics.
- Perform a comparative analysis against established baseline models (e.g., GAN architectures) to benchmark the DM's performance.

Work Package 5: Integration Analysis

Objective: To integrate all findings, analysis, and insights into the final thesis document.

Tasks:

- Compile the theoretical background, methodology, and results into cohesive chapters.
- Formulate conclusions answering the main research questions and identify potential avenues for future work.
- Prepare the reproducibility package (codebase, trained weights, dataset samples) for open-source publication.

3.5. Expected Contributions

This research is expected to yield several significant contributions to the field of Aerospace Engineering and Air Traffic Management:

1. **Methodological Innovation:** The development of a highly stable 1D U-Net Diffusion architecture operating directly within a Kinematic State Space for the high-dimensional, spatiotemporal generation of aircraft terminal trajectories. This streamlined, domain-specific approach provides an efficient alternative to over-parameterized image-based U-Nets and discrete transformer architectures, outperforming traditional avoiding instability issues from other baselines.
2. **Steerable Trajectory Generation:** By introducing advanced FiLM-based modulation into the diffusion framework, this research pioneers an innovative, steerable methodology for DMs. This allows operators to explicitly condition trajectory generation on specific operational requirements (e.g., aircraft type).
3. **Robust evaluation framework:** A rigorous statistical evaluation framework with self-evaluation capabilities encompassing distributional variety, temporal fidelity, overfitting check and downstream utility. Establishing best practices for assessing synthetic trajectory quality.
4. **Operational Impact:** The provision of highly realistic, varied, and flyable synthetic datasets will empower Air Navigation Service Providers (ANSPs) and safety researchers to execute robust Monte Carlo collision risk modeling and capacity analyses, overcoming the data access problems at regional airports.
5. **Full Reproducibility:** Open-source implementation and reproducibility package at <https://github.com/Mvile02/PI-DM>, including complete training code, pretrained model weights, pre-processed data, and figure generation scripts enabling full reproduction of all results.

Part II

Methodology

4

Methodology

This chapter outlines the methodological framework developed to extract, process, and generate aircraft trajectories. To establish a robust data pipeline and a highly stable kinematic generative architecture, the workflow is divided into five sequential stages. As illustrated in Figure 4.1, the progression spans from the initial extraction and clustering of real operational trajectories through the formulation and empirical training of the 1D U-Net diffusion model, concluding with a rigorous statistical evaluation framework.

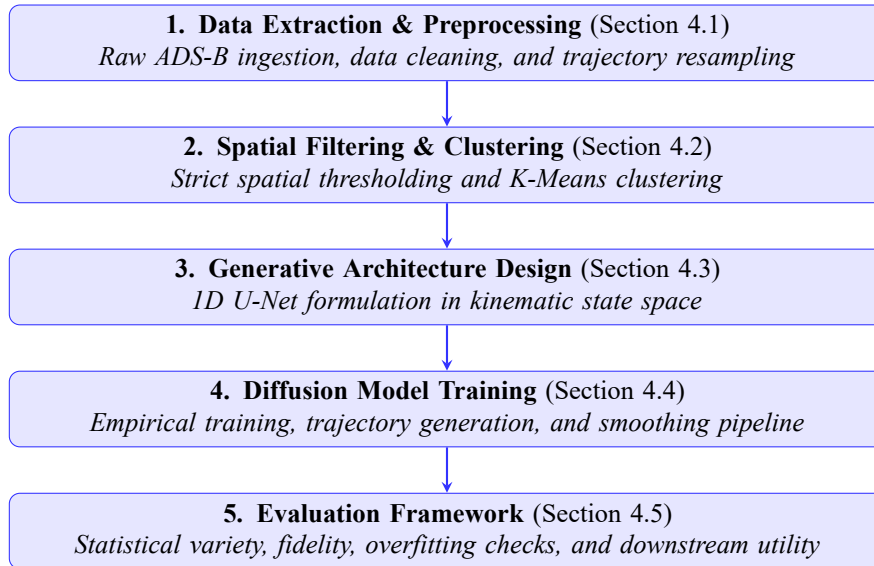


Figure 4.1: Methodological workflow outlining the progression from raw data extraction to synthetic trajectory evaluation.

4.1. Data Extraction and Preprocessing

The foundation of the generative model relies on high-quality empirical trajectory data. Historical aircraft surveillance data was sourced from the OpenSky Network. A modular, object-oriented architecture was implemented to ensure reliable and repeatable data processing.

A dedicated module was developed to parse and extract relevant trajectory information from the raw traffic data. Specifically, the module makes use of the *traffic* library to extract the 4 dimension landing trajectories for R14 in LSZH from October 1st to November 30 of 2019, filtering out holding patterns and go-arounds, already labeled in the dataset. The raw flight paths are processed into normalized tensors of shape $(N, 4, 200)$, where N represents the number of trajectories. Each trajectory is resampled and interpolated to a fixed length of 200 points. The four channels represent critical kinematic variables:

- Altitude [ft]
- Groundspeed [kts]
- Track Angle [$^{\circ}$]
- Time [s]

Additionally, the preprocessing pipeline includes metadata synchronization. By mapping ICAO24 transponder codes to specific aircraft typecodes (See Figure 4.2 legend) that can later serve as conditional inputs for the diffusion model.

To visualize the data more easily, the kinematics can be converted to coordinates (See Figure 4.3). The methodology is explained in Section 4.2.

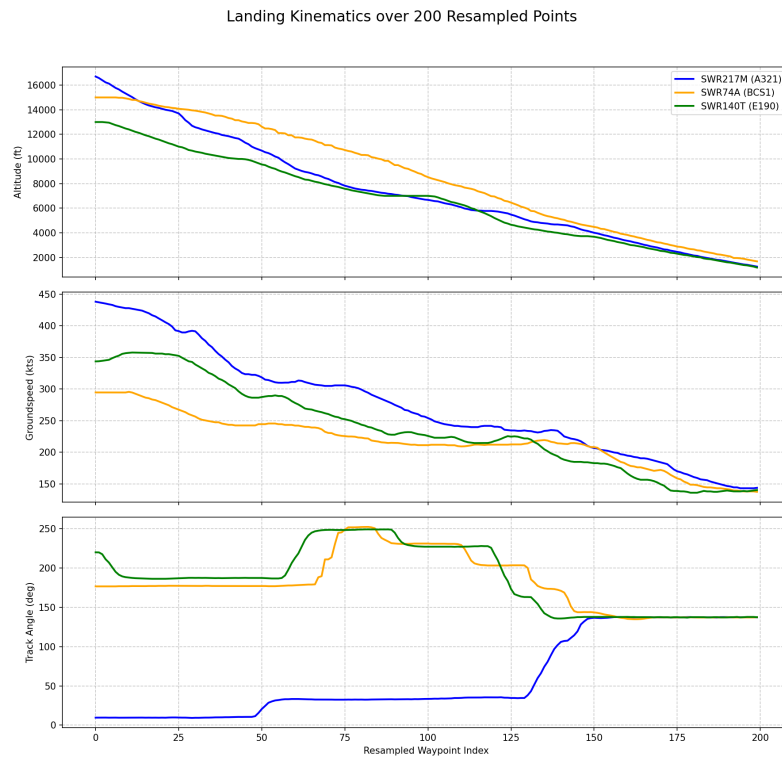


Figure 4.2: Visual representation of three of the raw extracted trajectories from the OpenSky Network.

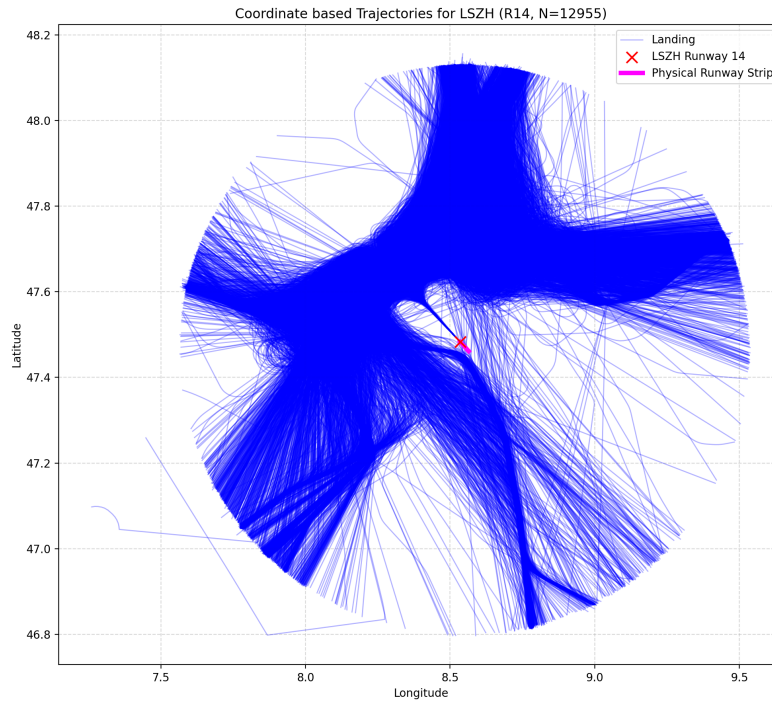


Figure 4.3: Coordinate based representation of the raw extracted trajectories from the OpenSky Network over the area of interest.

4.2. Spatial Filtering and Clustering

TMA's typically contain multiple Standard Terminal Arrival Routes (STARs) and approach procedures, resulting in highly varied and spatially distinct trajectory bundles (see Figure 4.3). While Diffusion Models are inherently resistant to mode collapse, training on an unconditioned, heavily multi-modal dataset can still result in the generation of physically averaged, interpolated paths that fall between two valid operational corridors. To prevent the synthesis of these operationally unviable trajectories and to establish a clean baseline for validating the core kinematic architecture, a spatial filtering pipeline was implemented to isolate distinct approach bundles.

To partition the dataset, the standard K -Means clustering algorithm was employed. K -Means is a foundational unsupervised machine learning technique that groups data into a predefined number of clusters (K) by iteratively assigning each point to the nearest cluster centroid (based on Euclidean distance [44]) and subsequently recalculating the centroid as the mean of its assigned points. This iterative process repeats until the assignments converge, effectively minimizing the within-cluster spatial variance [45].

In this thesis, the clustering methodology utilizes a geometric approach to map the kinematic sequences into physical space before the K -Means algorithm is applied:

1. **Dead Reckoning Coordinate Reconstruction:** The script reconstructs the local (x, y) coordinates (in meters) for every trajectory. This is achieved using Dead Reckoning backwards from the runway anchor (e.g., RWY 14 at LSZH):
 - Groundspeed is converted to meters per second (m/s), and heading to radians.
 - The change in position $(\Delta x, \Delta y)$ at each time step is calculated using average velocity.
 - The vectors are cumulatively summed backwards from the runway anchor point to map the true physical path in space.

2. **Standard K-Means on Spatial Paths:** Once the physical (x, y) sequences are generated, they are flattened into a spatial tensor (200 x -coordinates + 200 y -coordinates). Standard K-Means clustering is then applied directly to this physical spatial tensor to find the 4 “golden centerlines” for the approach bundles¹.
3. **Strict Spatial Threshold Filtering:** To ensure the diffusion model receives clean datasets, trajectories are filtered based on their physical deviation from the identified cluster centerlines. Initially, a large threshold of 75,000 m was applied to observe the raw, overarching cluster assignments without strict boundaries. However, to isolate tight, cohesive approach bundles suitable for generative modeling, it was determined that a stricter threshold of 5,000 m is necessary. Flights with a mean Euclidean deviation exceeding the 5,000 m threshold are discarded, and the remaining tight approach bundles are saved into individual cluster files. A comparison of these filtering thresholds is shown in Figure 4.4.

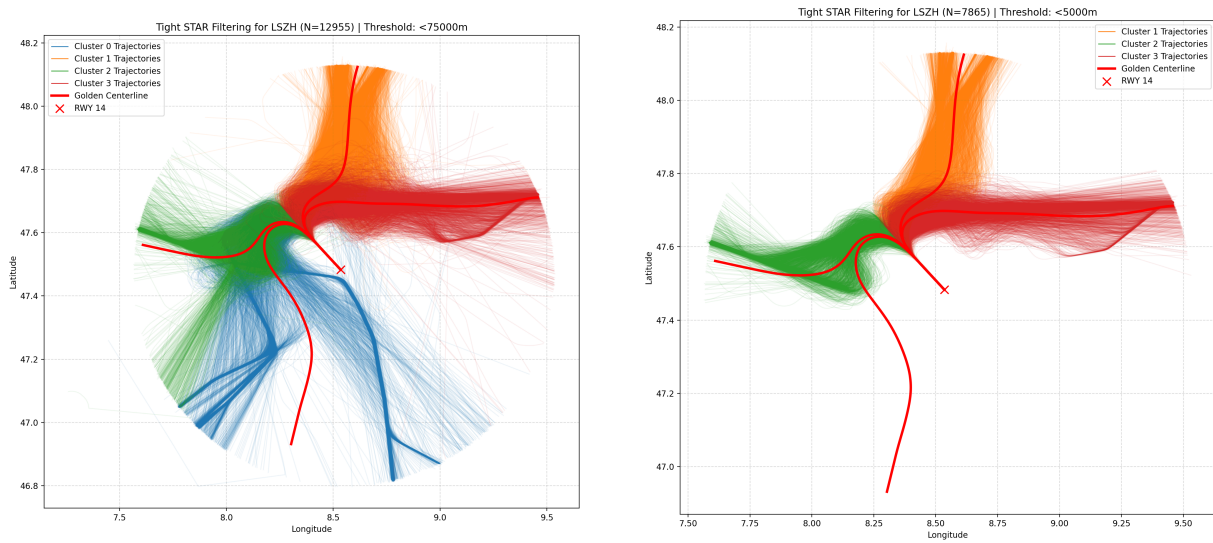


Figure 4.4: Comparison of spatial threshold filtering. Left: Clustering results with a relaxed spatial threshold of 75,000 m. Right: Refined clustering results with a strict spatial threshold of 5,000 m, isolating the core approach bundles and discarding outliers.

4.3. Generative Architecture: 1D U-Net in Kinematic Space

4.3.1. Learned Latent Spaces vs. Deterministic State Spaces

A prominent approach within the generative domain is the use of Latent Diffusion Models (LDMs). LDMs essentially combine a standard Diffusion Model with a preliminary Variational Autoencoder; the VAE first compresses high-dimensional data into a smaller continuous or discrete latent space, where the diffusion process is then executed. This compression step is widely adopted in image and video generation because it drastically reduces the computational cost and memory footprint required to run diffusion over millions of pixels. However, aircraft trajectories are relatively low-dimensional time series (e.g., sequences of 200 points with only four features). For such data, this unnecessary abstraction introduces training instability and bottleneck artifacts that often wash out high-frequency kinematic maneuvers. Indeed, recent work by Larsen [3] comparing Latent Diffusion against direct Diffusion Models concluded that direct diffusion performs superiorly, avoiding the effect size reduction caused by latent compression. Following this paradigm, an emerging alternative is the projection of trajectories into a Deterministic Kinematic State Space. By

¹Manually set to 4 based on Figure 4.3 visualization and subjective opinion.

extracting rotationally and translationally invariant kinematic state vectors (e.g., track angle, groundspeed, altitude) directly from the raw ADS-B surveillance data, the trajectories are inherently standardized into a domain-agnostic feature space. This achieves the core benefits of spatial abstraction—avoiding geographic leakage and isolating universal flight dynamics—without the computational overhead, smoothing artifacts, or instability of training an autoencoder. Therefore, **this thesis opts for a direct Diffusion Model operating purely in the kinematic state space** rather than relying on a preliminary VAE architecture.

4.3.2. Denoiser Architecture: 1D Convolutional U-Net

Within the direct diffusion paradigm, a critical architectural debate centers on the choice of denoiser: **Transformers vs. 2D U-Nets vs. 1D Convolutions**. While recent works like the one from Larsen et al. adapt 2D U-Nets borrowed directly from image generation [3], these adaptations rely on massive embedding blocks and computationally expensive Self-Attention layers that treat 1D time series as spatial images. Similarly, Transformer-based architectures rely inherently on these Self-Attention mechanisms, which operate by computing a dense correlation matrix where every point in the sequence attends to every other point simultaneously. While Transformers are highly effective at capturing long-range dependencies, the computational and memory complexity of their attention operations scales quadratically ($O(N^2)$) with the sequence length N , leading to significant overhead for long trajectory generation. Alternatively, cross-domain applications in ground vehicle trajectories, such as the work by Sokolov et al. [43], have demonstrated that **1D Convolutional U-Nets (or simply 1D U-Nets)** provide a far more streamlined, native inductive bias for local temporal continuity. Consequently, this thesis adapts and scales their architecture to act as the generative backbone for multivariate 4D aircraft trajectories.

The implemented 1D U-Net architecture utilizes a hierarchical downsampling and upsampling structure comprising exactly **three compressing steps** in the encoder, each of them called Residual Blocks (Residual Network (ResNet)). Therefore, the input trajectory sequence of length L is sequentially compressed to $L/2$, $L/4$, and finally down to a compact bottleneck representation of $L/8$. This significant $8\times$ compression factor efficiently expands the network’s receptive field, allowing the convolutions at the bottleneck to capture overarching global trends (such as the entire continuous descent profile), while the symmetric upsampling decoder (utilizing linear interpolation and skip-connections) reconstructs the high-frequency localized maneuvers (see Figure 4.5).

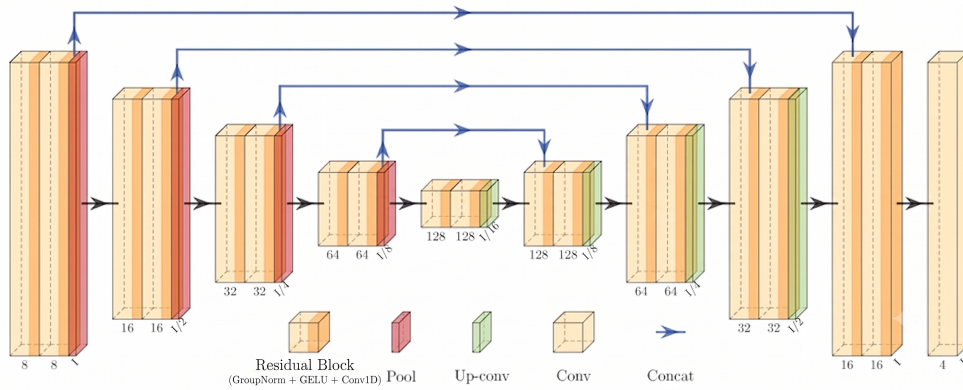


Figure 4.5: Example of a 1D UNet architecture with 4 compressing steps. Note that the feature channel is doubled while the feature size is halved after passing one layer of the encoder. The decoder mirrors this process, halving the feature channel and doubling the feature size after each layer. The blue arrows represent the skip connections, transferring the local details. Based on [46].

An architectural novelty introduced in this thesis to maximize computational efficiency and completely bypass the quadratic bottleneck of self-attention is the injection of a global absolute positional coordinate

axis (a normalized tensor spanning from -1.0 to 1.0 in L steps) directly into the input tensor. While standard CNNs lack temporal awareness, concatenating this physical sequence timeline as an explicit additional input channel provides the fully-convolutional network with absolute global temporal awareness (e.g., inherently learning that altitude must be high at -1.0 and near zero at 1.0) without relying on computationally expensive Transformers.

Crucially, this architecture inherently learns the strict physical correlations between different kinematic variables. While a 2D image convolution slides across spatial pixels and processes RGB colors as separate channels, a 1D sequence convolution slides across the temporal axis and processes the kinematic variables (e.g., altitude, true airspeed, heading) as its channels. At every temporal step, the convolutional kernel computes a weighted sum across all channels simultaneously. This mathematical formulation means the network explicitly evaluates altitude and velocity concurrently, naturally allowing the filter weights to encode aerodynamic laws—for instance, learning that a steep negative gradient in the altitude channel (a dive) must strongly correlate with an acceleration in the velocity channel. This concurrent channel processing ensures that the multivariate trajectories generated by the model remain physically realistic.

4.3.3. Conditional Integration via FiLM

Another significant architectural novelty of this implementation is the mechanism used to guide the generative process across diffusion timesteps, integrating external conditioning (e.g., aircraft type, airport, and meteorological weather). Rather than concatenating conditioning vectors into the input or relying on cross-attention, the model employs **Feature-wise Linear Modulation (FiLM)** [42] natively embedded at every single convolutional block (ResNet). While the FiLM architecture is fully programmed and mathematically prepared to process the complex concatenated embedding of aircraft type, target airport, and weather conditions, the empirical generation experiments conducted in this thesis focus exclusively on the aircraft type constraint, leaving the integration of multi-airport generation and weather processing for future work.

In the implementation, the current diffusion timestep and the target operational conditions are each processed and converted into simple vector embeddings. These embeddings are then summed together to form a unified global embedding context. At every residual block throughout the encoder, bottleneck, and decoder, this global embedding is projected via a linear layer to generate condition-dependent scale (s_c) and shift (b_c) parameters. FiLM then applies an affine transformation directly to the intermediate normalized feature maps ($\mathbf{f}$) of the 1D convolutions:

$$\mathbf{f}_{new} = s_c \odot \mathbf{f} + b_c \quad (4.1)$$

The exact structural flow of this conditioning process is visualized in Figure 4.6.

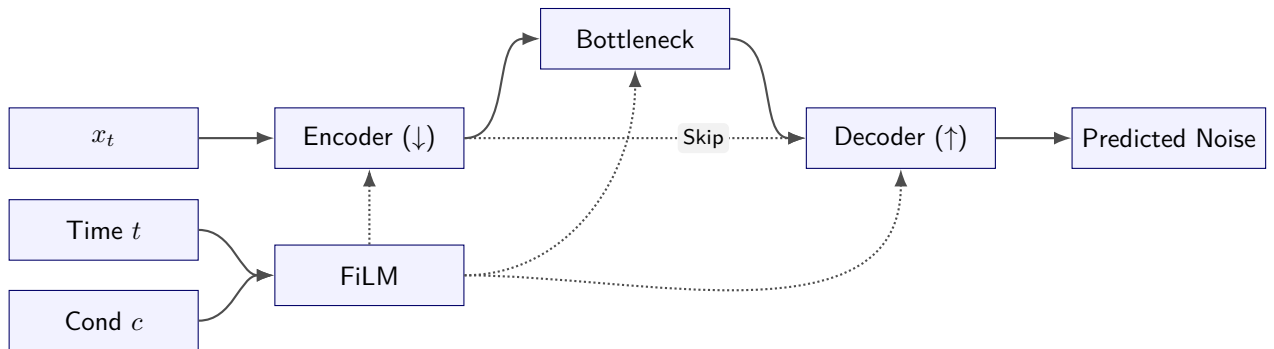


Figure 4.6: Architecture diagram demonstrating the injection of Time and Conditional embeddings into the 1D U-Net via Feature-wise Linear Modulation (FiLM). The unified global embedding vector modulates the spatial features across every hierarchical depth level of the denoiser.

Because this mathematical modulation occurs at every depth level of the U-Net, the global diffusion timeline and the specific operational constraints are explicitly forced into every hierarchical layer of the trajectory generation, steering the network to produce aerodynamic profiles that strictly match the requested scenario without the massive memory overhead of concatenating text into a Transformer’s input sequence.

4.3.4. Score-Based Diffusion Formulation

In this section, the mathematical formulation of the diffusion process is presented, detailing both the training and sampling phases.

1. Forward Diffusion Process & Training Objective: As seen in Section 2.6.4, the first step is the forward diffusion process, which incrementally adds noise to the data over T_{diff} time steps. From Equation 2.8, it is known that the encoder follows:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (4.2)$$

where $\bar{\alpha}_t = \prod_{s=1}^{T_{diff}} (1 - \beta_s)$, and β_t follows a linear schedule from $\beta_1 = 0.0001$ to $\beta_{T_{diff}} = 0.02$ [43]. This specific linear scaling is chosen to perfectly balance final noise corruption (ensuring $\mathbf{x}_{T_{diff}} \approx \mathcal{N}(0, \mathbf{I})$) while maintaining smooth transitions between discrete steps so the neural network can mathematically learn to reverse them.

During training, the model parameters θ are optimized to minimize the standard simplified variational lower bound objective ($\mathcal{L}_{\text{simple}}$) seen previously (Equation 2.10), which trains the network to predict the exact noise that was added at step t . The complete loss function can be defined, adding the conditioning variables $\mathbf{c}$ (aircraft type, airport, weather), as follows:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0, \epsilon, t, \mathbf{c}} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t, \mathbf{c}) \right\|^2 \right] \quad (4.3)$$

Where ϵ_{θ} is the 1D U-Net denoiser. The complete training procedure is summarized in Algorithm 1.

Algorithm 1: Conditional Diffusion Training

Data: Training dataset of kinematic trajectories $\mathcal{D}$, Sequence length L

Result: Optimized denoiser network parameters θ

Define absolute positional axis: $\mathbf{pos} = \text{linspace}(-1.0, 1.0, L)$;

repeat

$\mathbf{x}_0 \sim q(\mathbf{x}_0)$ from $\mathcal{D}$;

$\mathbf{c} \sim p(\mathbf{c})$ (Conditioning variables);

$t \sim \text{Uniform}(\{1, \dots, T_{diff}\})$;

$\epsilon \sim \mathcal{N}(0, I)$;

 Compute noisy trajectory: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$;

 Concatenate positional axis: $\mathbf{x}_t^{\text{in}} = \mathbf{x}_t \oplus \mathbf{pos}$;

 Take gradient descent step on;

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\mathbf{x}_t^{\text{in}}, t, \mathbf{c}) \right\|^2$$

until *loss improvement* $< 10^{-5}$ for 100 consecutive epochs;

2. Reverse Sampling (Generation): Now that the model is trained and has computed the θ parameters, it is capable of synthesizing new trajectories from pure noise $\mathbf{x}_{T_{diff}} \sim \mathcal{N}(0, \mathbf{I})$ by iteratively traversing a reverse Markov chain during the denoising sampling phase. To do so, the model must sample from the reverse trajectory distribution (see Equation 2.9).

Following the standard discrete Denoising Diffusion Probabilistic Model (DDPM) formulation [39], the covariance Σ_{θ} is fixed to an untrained time-dependent constant $\sigma_t^2 \mathbf{I}$ (where $\sigma_t^2 = \beta_t$). Crucially,

Ho et al. demonstrate that the most optimal parameterization for the mean μ_θ is to predict the forward process posterior mean, which can be algebraically rewritten to directly utilize the neural network’s noise prediction ϵ_θ (for detailed explanation, see [39]):

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t \oplus \mathbf{pos}, t, \mathbf{c}) \right) \quad (4.4)$$

Because sampling from a Gaussian $\mathcal{N}(\mu, \sigma^2)$ is computationally equivalent to computing the mean μ and adding standard stochastic noise $\mathcal{W}$ scaled by the standard deviation σ , the probabilistic definition in Equation 2.9 can be expressed as the following discrete iterative update step utilized to generate the sequence $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t \oplus \mathbf{pos}, t, \mathbf{c}) \right) + \sigma_t \mathcal{W} \quad (4.5)$$

where α_t and $\bar{\alpha}_t$ are defined by the forward noise variance schedule, and $\mathcal{W} \sim \mathcal{N}(0, \mathbf{I})$ introduces the required stochasticity scaled by the variance $\sigma_t^2 = \beta_t$. This discrete step is exactly what physically morphs the pure Gaussian noise back into a flyable sequence, heavily steered by the FiLM-injected conditions $\mathbf{c}$. This iterative sampling process is detailed in Algorithm 2.

Algorithm 2: Conditional Denoising Generation (Sampling)

Data: Target conditioning variables $\mathbf{c}$, Sequence length L

Result: Generated synthetic trajectory $\mathbf{x}_0$

Define absolute positional axis: $\mathbf{pos} = \text{linspace}(-1.0, 1.0, L)$;

$\mathbf{x}_{T_{diff}} \sim \mathcal{N}(0, \mathbf{I})$;

for $t = T, \dots, 1$ **do**

$\mathcal{W} \sim \mathcal{N}(0, I)$ if $t > 1$, else $\mathcal{W} = 0$;

 Concatenate positional axis: $\mathbf{x}_t^{\text{in}} = \mathbf{x}_t \oplus \mathbf{pos}$;

$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t^{\text{in}}, t, \mathbf{c}) \right) + \sigma_t \mathcal{W}$;

end

return $\mathbf{x}_0$;

4.4. Diffusion Model Training

With a clean, clustered dataset established and the generative architecture mathematically formulated, the next phase involved the empirical training of the model.

In the theoretical formulation (Algorithm 1), the learning process is abstracted as a generic gradient descent step (∇_θ). In practice, executing this step with vanilla Stochastic Gradient Descent (SGD):

$$\theta_{new} = \theta - \eta \nabla_\theta \quad (\eta = \text{learning rate}) \quad (4.6)$$

is inefficient. Instead, Adam [47] serves as the computational engine that intercepts these gradients and computes individual adaptive learning rates for the millions of neural network weights by maintaining running averages of their first and second moments. Recent mathematical analyses have rigorously proven optimal convergence rates for Adam across a large class of stochastic optimization problems, that is why the model was empirically trained using Adam [47].

For this implementation, the optimizer’s internal momentum parameters were set to standard literature defaults ($\beta_1 = 0.9$, $\beta_2 = 0.999$), alongside a global learning rate of 10^{-4} and a batch size of 32. The diffusion process employed $T_{diff} = 1,000$ timesteps. To prevent overfitting and ensure computational efficiency, the training loop was optimized to utilize convergence-based early stopping rather than relying on a fixed number of epochs (although an upper limit of 20,000 epochs was set). Specifically, training was halted if the $\mathcal{L}_{\text{simple}}$ loss failed to improve by a minimum delta of 10^{-5} for a patience period of 100 consecutive epochs.

4.4.1. Cluster 1 Validation and Trajectory Filtering

To establish a controlled baseline and validate the U-Net architecture without the overwhelming complexity of the massive multimodal dataset, training commenced with the isolated *Cluster 1* (see Figure 4.7).

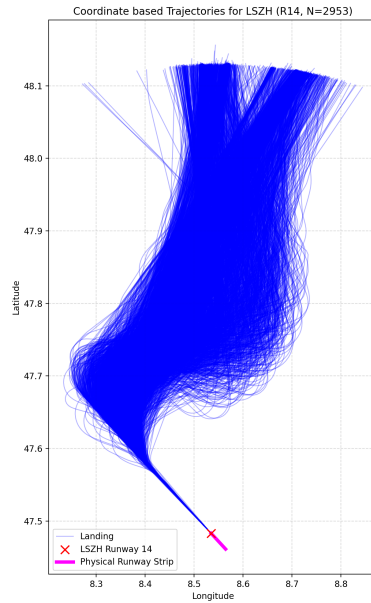


Figure 4.7: Isolated Cluster 1 (from Figure 4.4) after applying the 5,000 m spatial threshold.

Once trained exclusively on this cluster, the base model proved capable of generating synthetic trajectories that effectively mimicked the spatial and kinematic distributions of the training subset.

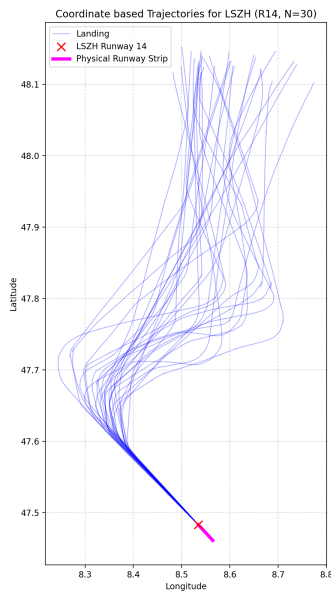


Figure 4.8: Generated coordinate trajectories ($N = 30$).

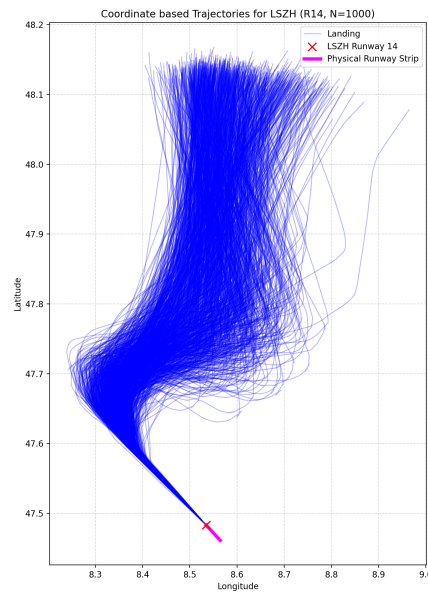


Figure 4.9: Expanded generation of synthetic coordinate trajectories ($N = 1000$ samples).

Figure 4.10: Raw synthetic trajectories generated by the model.

However, during the initial generation of these synthetic trajectories, it was observed that some raw

generated paths exhibited underderivability, physically manifesting as violent 180° switchbacks—where the trajectory sharply turns back on itself before correcting to its original path (as seen in Figure 4.10).

The root cause of this phenomenon lies in the representation of the aircraft’s track angle within the training dataset. Track angles are measured in degrees and bounded between 0° and 360° . While a maneuver crossing true North (e.g., smoothly turning from 359° to 1°) represents a minimal and continuous 2° physical turn, it introduces a severe mathematical discontinuity—a sudden step-function drop of 358° in the numerical array.

Because the neural network optimizing the loss function operates under the assumption of continuous numerical sequences, it inherently struggles to reconstruct this massive high-frequency step function. The diffusion model attempts to smooth the transition over several timesteps, generating intermediate values that bridge the 359° to 1° gap. When these corrupted, intermediate track angles are subsequently integrated to compute X/Y spatial coordinates via Dead Reckoning, the trajectory undergoes severe, artificial zig-zags or switchbacks.

To resolve this issue, a post-processing filtering pipeline was implemented. First, the generated track angles were mathematically unwrapped to eliminate the 360° boundary cliffs. This algorithm detects sequential jumps larger than 180° and conceptually “unwraps” the circle into a flat, continuous line (e.g., transforming a physical heading sequence recorded as $[356, 358, 0, 2, 4]$ into a mathematically smooth $[356, 358, 360, 362, 364]$). Subsequently, a Median filter [48] was applied to remove extreme outliers without blurring sharp legitimate turns, followed by a Savitzky-Golay filter [49] to ensure the resulting curves were smooth and fully derivable.

While post-processing effectively mitigates the issue, a more robust and fundamentally native solution exists for future iterations. By transforming the raw track angle into two continuous trigonometric features ($\sin(\text{track})$ and $\cos(\text{track})$) prior to training, the 360° boundary discontinuity would be completely eliminated. The neural network could then seamlessly learn the continuous transitions, recovering the angle post-generation using the two-argument arctangent function. Although recognized as the optimal approach, this fundamental architectural data change was deferred in the current implementation due to strict timing constraints.

4.4.2. Full Conditional Dataset Scaling

Following the successful validation of the core generative architecture and the establishment of the post-processing filtering pipeline on *Cluster 1*, the training paradigm was scaled to encompass the full, unclustered conditional dataset, keeping the spatial filtering of 5000m (see Figure 4.11).

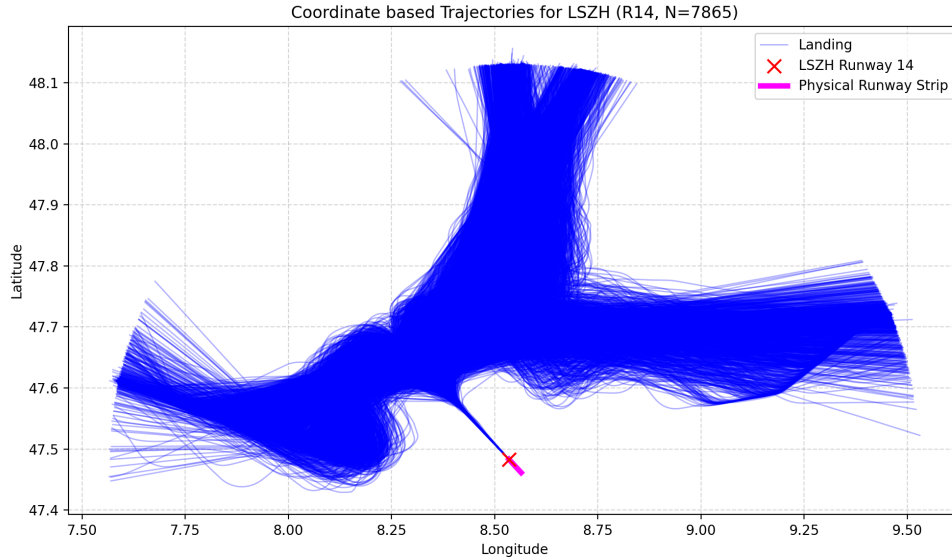


Figure 4.11: The real empirical coordinate based trajectories from the three main spatial clusters shown together.

Because the full dataset contains a massive, multi-modal distribution of trajectories flown by different aircraft types, a standard unconditional diffusion model would fail to capture the specific kinematic bounds of each category, resulting in generic, averaged flight paths. To resolve this, the network leveraged the FiLM conditioning mechanism previously detailed in Section 4.3.3. By directly embedding the `aircraft_type` identifier into the network’s feature maps at every depth level, the single generative model was mathematically guided to learn the distinct aerodynamic regimes of the entire dataset simultaneously.

4.5. Evaluation Framework

The validation of synthetic trajectories requires a multi-layered evaluation framework capable of rigorously quantifying the generative model’s performance. To achieve this, the generated trajectories are subjected to a comprehensive suite of statistical metrics specifically designed to assess distribution diversity, temporal fidelity, data memorization (overfitting), and downstream utility.

Variety (Diversity): To verify that the generative model captures the full operational envelope of the training data and avoids mode collapse, the diversity of the generated trajectories is assessed. This is achieved via dimensionality reduction using Principal Component Analysis (PCA) [50] and t-Distributed Stochastic Neighbor Embedding (t-SNE) [51]. Trajectories are first preprocessed into a lower-dimensional representation (e.g., by computing temporal means or flattening features) and then projected onto a two-dimensional subspace. Plotting the projected points of both historical and synthetic datasets allows visual inspection of the spatial and distributional overlap, ensuring that the generated flights span the entire distribution of the real data.

Fidelity (Similarity): Fidelity metrics evaluate how closely the individual synthetic trajectories mimic the structural characteristics and multi-dimensional distribution of the real flights.

- **LSTM Discriminator (Discriminative Score):** To evaluate the temporal and dynamic realism of the trajectories, a sequence-to-sequence neural network is employed as a discriminator. A LSTM-based binary classifier is trained to distinguish between real trajectories (label 1) and synthetic tra-

jectories (label 0). The performance of the classifier is summarized by the Discriminative Score, defined as:

$$\text{Discriminative Score} = |\text{Accuracy} - 0.5| \quad (4.7)$$

where Accuracy represents the classifier's test accuracy. An ideal score of 0.0 indicates that the generated trajectories are statistically indistinguishable from real ones, as the discriminator cannot perform better than random guessing.

- **Energy Distance [52]:** To quantify the macro-level similarity between the empirical distributions of real and synthetic trajectories, the mathematical energy distance is calculated. For two independent random vectors $\mathbf{X}$ and $\mathbf{Y}$ representing real and synthetic trajectories respectively, the energy distance is defined as:

$$D^2(F_{\mathbf{X}}, F_{\mathbf{Y}}) = 2\mathbb{E}[\|\mathbf{X} - \mathbf{Y}\|] - \mathbb{E}[\|\mathbf{X} - \mathbf{X}'\|] - \mathbb{E}[\|\mathbf{Y} - \mathbf{Y}'\|] \quad (4.8)$$

where $\mathbf{X}'$ and $\mathbf{Y}'$ are independent copies of $\mathbf{X}$ and $\mathbf{Y}$. In practice, to handle high-dimensional sequences and avoid computational bottlenecks, the trajectories are flattened and scaled, and the distance is computed over bootstrapped samples. A lower energy distance (approaching 0.0) indicates high similarity between the probability distributions.

Memorization Check (Overfitting): Generative models—especially those operating in high-dimensional spaces with limited data—are highly susceptible to overfitting, where the model simply memorizes and carbon-copies the training data rather than learning to synthesize novel trajectories. To rigorously detect this phenomenon, a 1-Nearest Neighbor (1-NN) Distance Analysis [45] is implemented. The methodology computes two distinct distance distributions:

1. **Real-to-Real Distance:** The Euclidean distance from each real trajectory in a normalized subsample to its closest independent counterpart within the real dataset.
2. **Synthetic-to-Real Distance:** The Euclidean distance from each generated trajectory to its closest real trajectory in the training set.

If the *Synthetic-to-Real* distance distribution is significantly skewed toward zero or falls heavily below the *Real-to-Real* baseline distribution, it indicates severe training data memorization. Conversely, if the distributions overlap closely, the generative model has successfully generalized the underlying state space dynamics and is producing mathematically novel flight paths without overfitting.

Utility (Usefulness): Utility metrics assess the downstream value of the generated data, determining whether synthetic trajectories can serve as viable training data for other machine learning models. This is evaluated using the TSTR methodology for time-series forecasting. An LSTM Regressor is trained entirely on synthetic trajectories to predict future states (specifically, final groundspeed and altitude given earlier sequence steps). The trained model's forecasting performance is evaluated using Mean Absolute Error (MAE) on a test set of real historical trajectories. This downstream MAE is compared against a Train on Real, Test on Real (TRTR) baseline model trained on historical data, yielding the Downstream MAE Ratio [53, 54]:

$$\text{Downstream MAE Ratio} = \frac{\text{MAE}_{\text{TSTR}}}{\text{MAE}_{\text{TRTR}}} \quad (4.9)$$

A ratio close to 1.0 indicates that models trained on synthetic trajectories achieve generalization performance comparable to those trained on real data, demonstrating high downstream utility.

4.5.1. Self-Diagnostic Pipeline

To make sure the evaluation metrics themselves are valid and sensitive to deficiencies in the generated trajectories, a self-diagnostic pipeline is established. This pipeline evaluates the sensitivity and correctness of the metrics under two baseline scenarios:

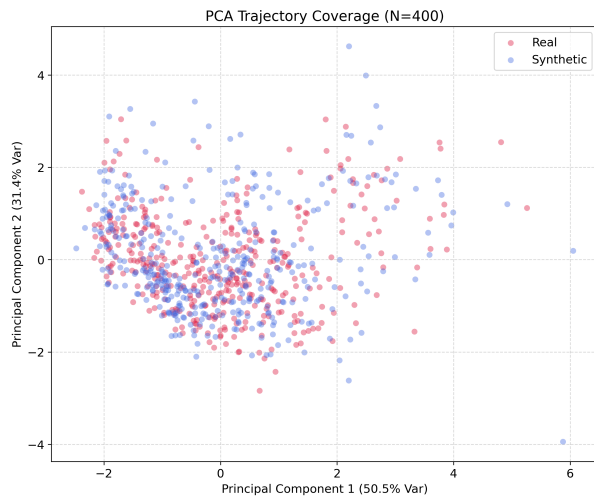
- **Perfect Baseline:** The reference real dataset is split in half and evaluated against itself, simulating an ideal generator. This scenario is expected to yield high PCA/t-SNE overlap, a discriminative score close to 0.0, and a TSTR/TRTR MAE ratio near 1.0.
- **Degraded Baseline:** One half of the real dataset is systematically corrupted (e.g., adding heavy heading noise, reducing groundspeed to simulate stall, introducing high-frequency altitude noise, and shuffling sequence timesteps) and evaluated against the clean half. This scenario tests the metrics' sensitivity, expecting to yield clear separation in PCA/t-SNE, a discriminative score of 0.5 (accuracy of 100%), a significantly higher Energy Distance, and a dramatically spiked TSTR/TRTR MAE ratio.

This diagnostic suite ensures that the evaluation pipeline is capable of reliably identifying deficiencies in generated flight paths. The quantitative results of this self-diagnostic verification are summarized in Table 4.1 and Figures 4.12 and 4.13, clearly demonstrating the metrics' sensitivity to corrupted dynamics and providing exact numerical bounds for the empirical minimum and maximum expected scores.

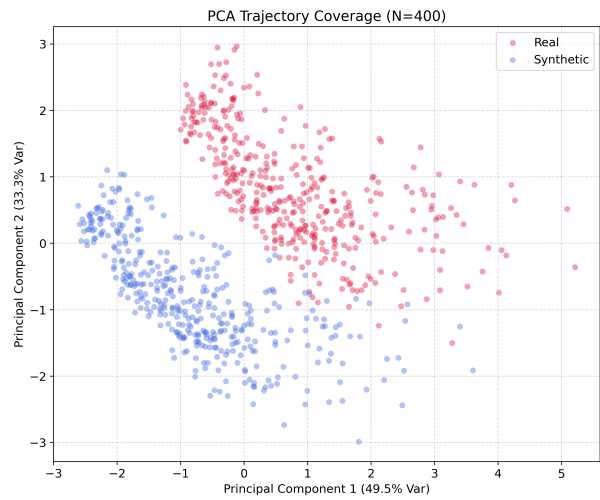
Table 4.1: Self-diagnostic pipeline results validating the sensitivity of the evaluation metrics. The Perfect Baseline evaluates the real dataset against itself, while the Degraded Baseline evaluates it against a systematically corrupted subset.

Metric	Perfect Baseline	Degraded Baseline
Average Energy Distance	2.139	5.742
LSTM Discriminator Test Accuracy	56.25%	100.00%
Discriminative Score	0.063	0.500
Downstream MAE Ratio (TSTR/TRTR)	0.995	5.757

Due to variance in finite sampling sizes, it is important to note that the Energy Distance for the perfect baseline yields an empirical minimum of approximately 2.14 (rather than a theoretical 0.0), while the maximum measured for completely disjoint distributions was 5.74. Furthermore, the TSTR/TRTR MAE ratio spiked to 5.75 in the degraded scenario, successfully penalizing the loss of physical realism and demonstrating that models trained on corrupted trajectories completely lose their downstream predictive utility.

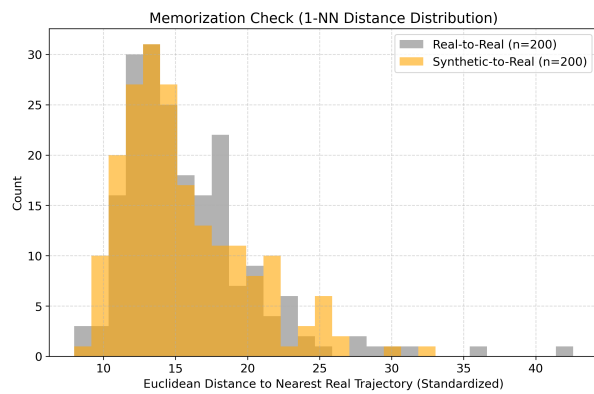


Perfect Baseline (Real vs Real)

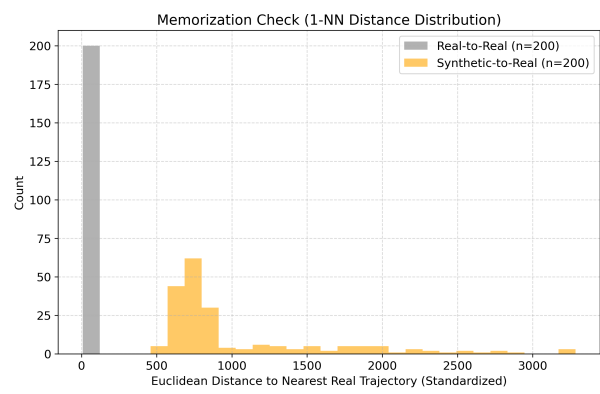


Degraded Baseline (Real vs Corrupted)

Figure 4.12: Self-diagnostic PCA coverage results, validating metric sensitivity. The perfect baseline showcases complete distributional overlap, while the corrupted baseline exhibits clear separation.



Perfect Baseline



Degraded Baseline

Figure 4.13: Self-diagnostic 1-NN Distance Analysis for memorization checks. Note the expected shifts in the intra-set and inter-set nearest neighbor distances when trajectories are fundamentally disjoint.

Part III

Results & Discussion

5

Results

This chapter presents the empirical findings of the implemented 1D U-Net Diffusion Model. Section 5.1 details the trajectory generation results, beginning with the successful post-processing of a single cluster before scaling to the full multi-modal generation of the three main approach bundles. Following the generation phase, Section 5.2 provides a comprehensive statistical evaluation of the synthetic data, analyzing its visual diversity, temporal fidelity, downstream usefulness, and confirming the absence of training data memorization.

5.1. Trajectory Generation Results

5.1.1. Cluster 1 Post-Processing

As detailed in the methodology, the initial synthetic trajectories generated for a single cluster (*Cluster 1*) exhibited artificial switchbacks due to angular discontinuities. Following the application of the post-processing pipeline—which incorporates phase unwrapping, Median filtering, and Savitzky-Golay smoothing—these artifacts were completely removed. Figure 5.1 illustrates the final synthetic trajectories for Cluster 1, successfully capturing the expected spatial distribution and continuous kinematics.

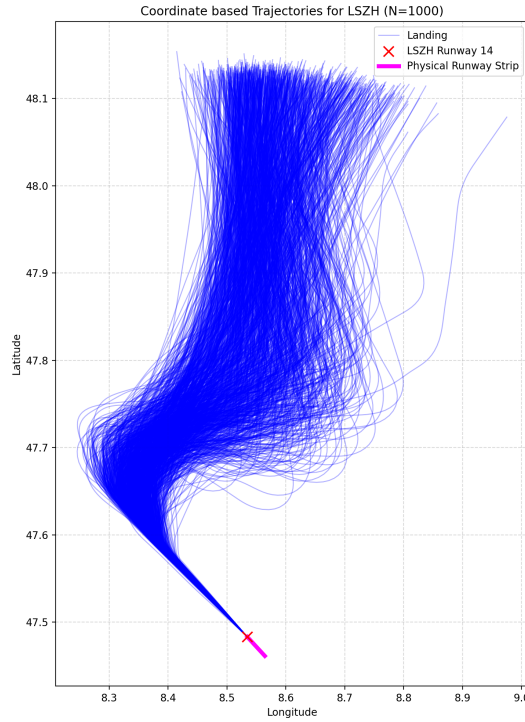


Figure 5.1: 1000 synthetic trajectories for Cluster 1 after post-processing filtering, successfully removing the artificial switchback artifacts.

As it can be seen, the model successfully learned the underlying kinematic principles of flight, generating realistic trajectories that closely resemble the real data (Figure 4.7). Nevertheless, it is important to note that there is a slight deviation at the entering of the TMA, where the original data exhibits two more clear distinct entry points, while the generated one shows a more continuous distribution. As the data conforming this particularity is relatively scarce compared to the rest of the dataset, this behavior is expected and represents a reasonable generalization. Furthermore, since the distinction between these entry points is not particularly sharp in the original dataset either, this smoothing can be viewed positively as evidence that the model is not merely copying training trajectories but rather learning the underlying distribution and generating novel variations.

5.1.2. Full Conditional Generation

Following the successful generation and smoothing of trajectories for a single cluster, the model's capability was extended to generate trajectories covering the three main approach bundles simultaneously.

The resulting generation of 1000 synthetic trajectories corresponding to these combined clusters proved highly effective, successfully capturing the distinct spatial bundles without visible mode collapse (Figure 5.2).

Moreover, looking at the kinematic representation of the generated trajectories (Figure 5.3), it is evident that the model has learned to produce realistic kinematic sequences that closely resemble the real data (Figure 4.2), preserving the noise characteristics inherent in the original dataset. This indicates that the model not only has captured the long-range dependencies across the entire trajectory but also maintains the fine-grained local variations, which are crucial for realistic flight path generation.

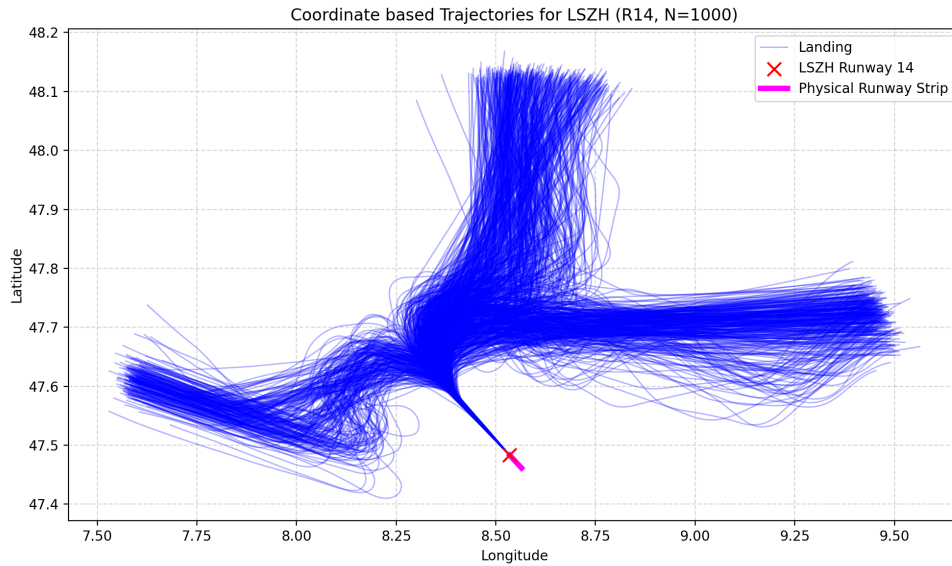


Figure 5.2: Corresponding coordinate based generated synthetic trajectories matching the three main clusters ($N = 1000$ samples).

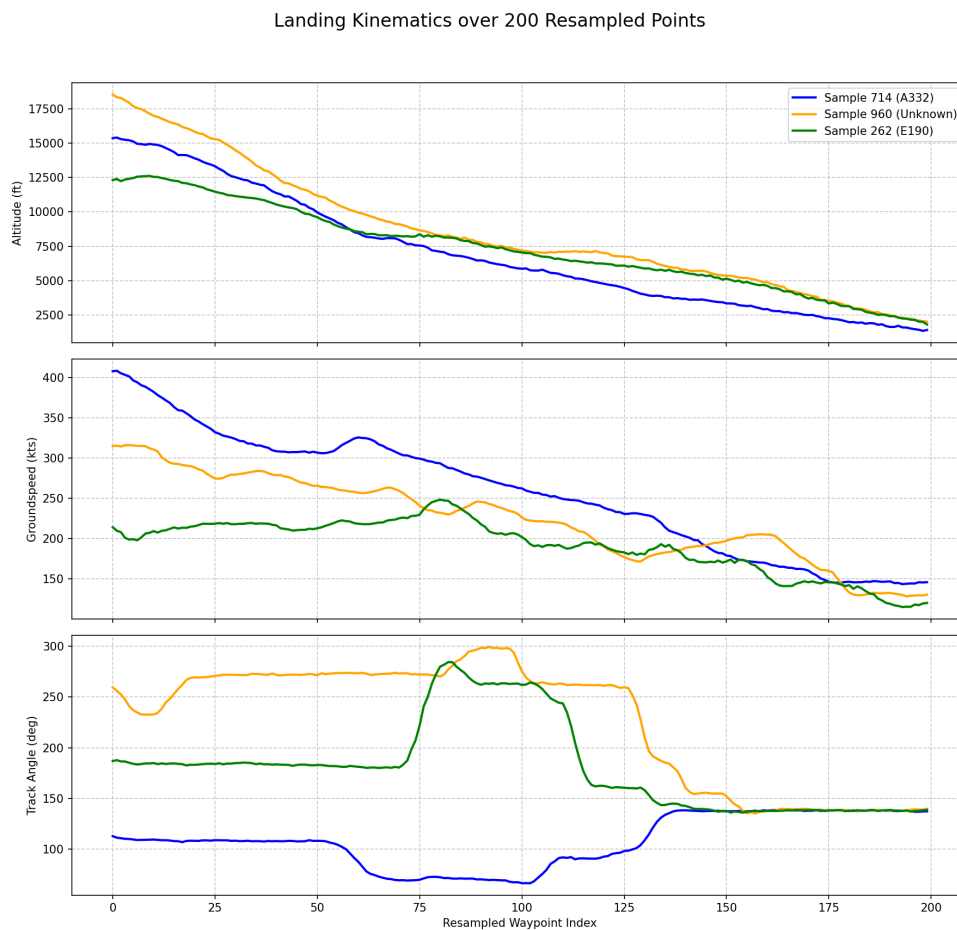


Figure 5.3: Representation of three random generated synthetic trajectories (kinematics).

5.2. Statistical evaluation

To quantitatively and qualitatively assess the fidelity of the generated trajectories, several evaluation metrics were applied. The visual diversity of the synthetic trajectories was compared against the real dataset using PCA and t-SNE, as shown in Figure 5.6. The PCA projection (Figure 5.4) reveals that the synthetic trajectories effectively cover the same principal variance directions as the real data, capturing the overarching spatial distributions without exhibiting mode collapse. Similarly, the t-SNE visualization (Figure 5.5) demonstrates a strong alignment in the local, non-linear manifold structure, confirming that the generated samples smoothly populate the same high-dimensional coordinate space as the real empirical flights.

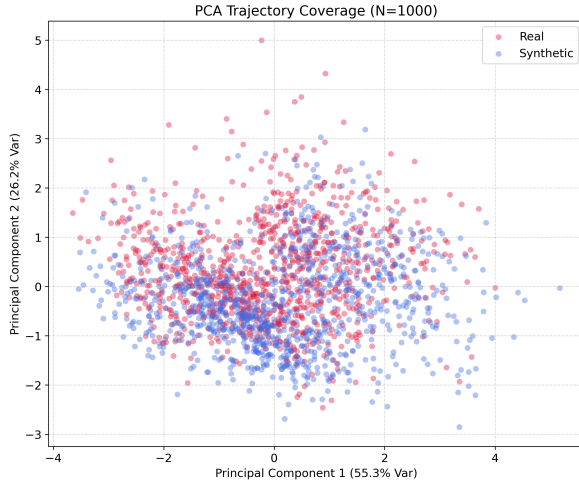


Figure 5.4: Visual diversity comparison using PCA.

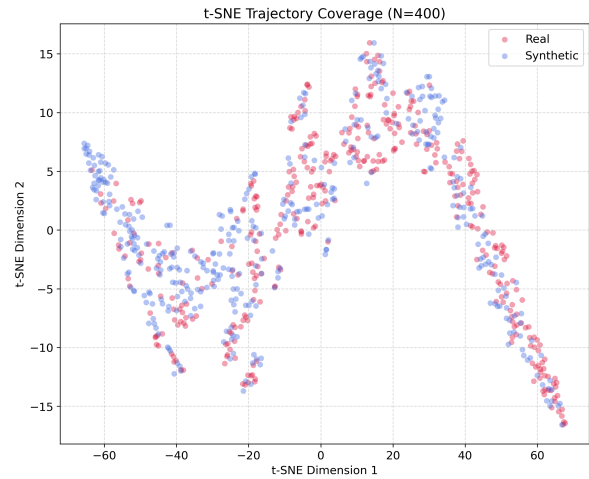


Figure 5.5: Visual diversity comparison using t-SNE.

Figure 5.6: Visual diversity plots evaluating the distribution of synthetic versus real trajectories.

Furthermore, a trajectory memorization check was conducted to ensure the generative model was not simply overfitting and replicating the training data. The 1-Nearest Neighbor (1-NN) distance analysis is presented in Figure 5.7. This plot illustrates the probability density of 1-NN Euclidean distances [44] for both real-to-real pairs (representing the natural density of the dataset) and synthetic-to-real pairs. The observable rightward shift in the synthetic-to-real curve compared to the real-to-real baseline indicates that the generated samples maintain a healthy distance from the training data. This lack of exact overlap confirms the model's ability to interpolate and synthesize entirely new flight paths rather than memorizing and duplicating existing ones.

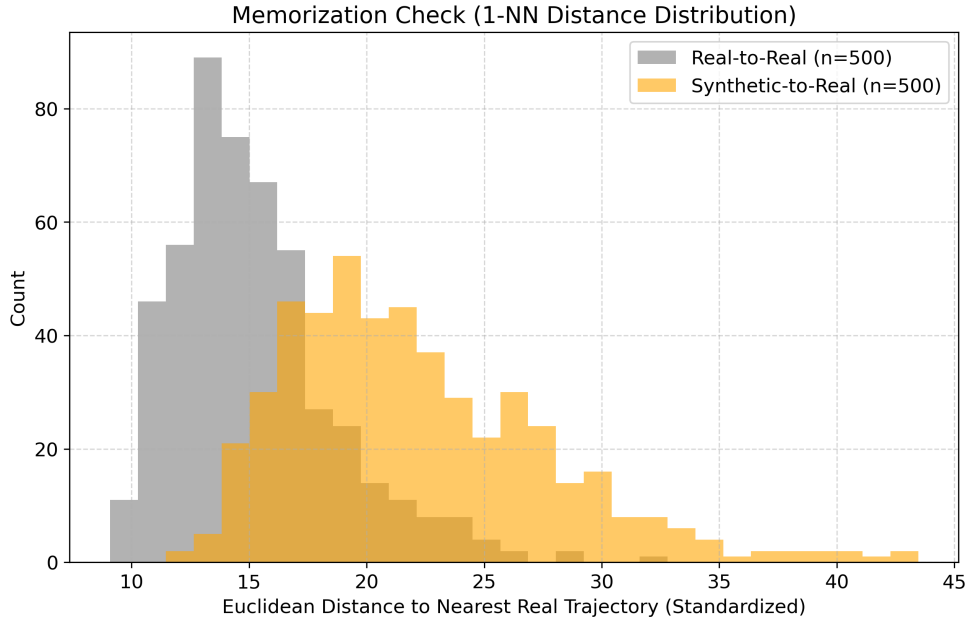


Figure 5.7: Trajectory memorization check analyzing the 1-NN distances to verify the absence of exact training copies.

The comprehensive results of the remaining evaluation tests are summarized in Table 5.1. To thoroughly understand the performance of the generative model, the metrics are analyzed across four key dimensions: distributional similarity, temporal fidelity, downstream usefulness, and data memorization.

Table 5.1: Summary of trajectory evaluation results.

Metric	Value
<i>Distributional Similarity</i>	
Average Energy Distance	1.44052
<i>Temporal Fidelity</i>	
LSTM Discriminator Test Accuracy	58.50%
Discriminative Score ($ \text{Accuracy} - 0.5 $)	0.0850
<i>Downstream Usefulness</i>	
TSTR Average MAE (Real Test Data)	0.74579
TRTR Average MAE (Real Test Data)	0.68891
Downstream MAE Ratio (TSTR/TRTR)	1.0826
<i>Trajectory Memorization</i>	
Median Real-to-Real Distance	14.4710
Median Synthetic-to-Real Distance	21.4939
Suspected Exact Training Copies	0.00%

First, the recorded **Average Energy Distance** of 1.44 falls well below the empirical perfect baseline evaluated during the diagnostic phase (≈ 2.14 , as detailed in Section 4.5.1), and is far below the distance evaluated for completely distinct distributions (5.74). This demonstrates that the generative model successfully learned the underlying spatial and kinematic distributions of the combined clusters, closely matching

the real data without producing disjoint or degenerate outputs.

Second, regarding **Temporal Fidelity**, the discriminator achieved a test accuracy of 58.50%, which translates to a discriminative score of 0.0850. As this score is remarkably close to the ideal 0.0 (random guessing), it demonstrates that the synthetic trajectories exhibit highly realistic temporal dynamics, making them mathematically nearly indistinguishable from real flight sequences.

Third, in terms of **Downstream Usefulness**, the TSTR model achieved a normalized Mean Absolute Error (MAE) of 0.746, closely approaching the TRTR baseline MAE of 0.689. The resulting MAE ratio of 1.083 provides strong evidence that the synthetic data accurately captures the essential physical relationships necessary for downstream machine learning forecasting tasks, serving as a highly effective substitute for real operational data.

Finally, the **Trajectory Memorization** check revealed that the median distance from synthetic samples to their nearest real counterpart (21.49) is actually greater than the median distance between real samples themselves (14.47). Most importantly, 0.00% of the generated samples fell below the similarity threshold that would flag them as exact training copies. This confirms that the model synthesizes novel, varied trajectories without overfitting or compromising the privacy of the original dataset.

In addition to the continuous kinematic features, ensuring that the synthetic dataset contains a realistic distribution of aircraft types is a crucial aspect of generative fidelity. During the inference phase, the generative model was explicitly conditioned by sampling target aircraft types according to the empirical probability distribution of the real dataset. Table 5.2 presents the relative frequencies of the ten most common aircraft types for both the real ($N = 7865$) and synthetic datasets ($N = 1000$), with the remaining tail of the distribution grouped into an “Others” category. The results demonstrate a remarkably close alignment between the real and synthetic categorical distributions. Rather than proving the model implicitly learned these frequencies, this alignment serves as a direct validation of the data generation pipeline; it confirms that the categorical sampling mechanism functions correctly, ensuring the final synthetic dataset accurately mirrors the operational traffic mix of the real-world airspace.

Table 5.2: Comparison of aircraft type distribution between real and synthetic trajectories. The ten most frequent types are shown individually, while the remainder are aggregated.

Aircraft Type	Real Dataset (%)	Synthetic Dataset (%)
A320	24.04	24.20
A319	12.18	11.70
E190	9.33	10.10
BCS3	7.81	7.20
A321	5.58	4.40
BCS1	3.95	3.60
A333	3.87	3.70
Unknown	3.74	4.90
B738	3.27	4.60
B77W	2.96	3.20
Others	23.27	22.40

5.3. Comparison with Reference Models

To contextualize the performance of the proposed 1D U-Net DM, its results are compared against a reference data-driven trajectory generation model: the TimeGAN model built by Wijnands et al. [2], previously

developed for the same dataset. Table 5.3 summarizes the available evaluation metrics across the two architectures.

Table 5.3: Performance comparison of the proposed 1D DM against the reference TimeGAN model.

Metric	1D DM (Proposed)	TimeGAN
Discriminator Accuracy	58.50%	60.80%
Discriminative Score	0.0850	0.1080
Downstream MAE Ratio (TSTR/TRTR)	1.083	0.995
Average Energy Distance	1.441	1.482

The comparison highlights the competitive performance of the 1D U-Net DM architecture against the TimeGAN baseline across all evaluated metrics. First, regarding temporal fidelity, the DM achieved a Discriminator Accuracy of 58.50% (yielding a Discriminative Score of 0.0850), performing slightly better than TimeGAN’s 60.80%. This indicates that the temporal dynamics generated by the diffusion process are marginally harder for the LSTM to distinguish from real trajectories.

In terms of downstream usefulness, both models demonstrate excellent results, acting as highly effective substitutes for real data in predictive tasks. The DM achieves a TSTR/TRTR MAE ratio of 1.083, performing very competitively alongside TimeGAN’s slightly closer ratio of 0.995.

Finally, the Average Energy Distance for the DM (1.441) is slightly lower than that of TimeGAN (1.482), indicating a marginal improvement in capturing the macro-level trajectory distributions.

While the statistical scores across all evaluated dimensions are remarkably similar, it is critical to note the fundamental architectural advantage of the proposed diffusion framework: its generative stability. By completely avoiding the training instability inherently associated with adversarial networks, the 1D U-Net DM captures the broader multi-modal spatial distributions of the dataset without suffering from the mode collapse frequently observed in GANs. Ultimately, this demonstrates that the DM is capable of achieving highly competitive, state-of-the-art statistical results while providing a significantly more stable and reliable methodology.

6

Discussion

A defining feature of the proposed methodology is the integration of FiLM conditioning to inject categorical metadata—specifically the `aircraft_type`—into the generative process. As demonstrated in the results, the generated dataset successfully replicated the varied empirical distribution of aircraft types found in the real world. However, no direct quantitative comparison against an unconditional baseline generation was performed. While an unconditional model could theoretically learn the global multimodal distribution of the trajectories, isolating the specific improvements brought by the conditioning is highly complex. The kinematic differences between aircraft types during approach (e.g., subtle variations in approach speeds or descent rates between a heavy wide-body aircraft like a B77W and a narrow-body like an A320) are relatively small compared to the macro-level spatial variance of the flight paths themselves. Capturing and proving these fine-grained differences would require an exhaustive, highly sensitive evaluation framework tailored specifically to aerodynamic performance, which falls outside the primary scope of this spatial and temporal generation study. Thus, the conditioning was evaluated based on its success in preserving categorical diversity rather than its marginal kinematic improvements over an unconditional baseline.

Another interesting phenomenon observed during generation was the model’s handling of circular variables. The initial synthetic trajectories exhibited artificial switchbacks due to angular discontinuities when the track angle crossed the $0^\circ/360^\circ$ boundary, as the network perceived 359° and 1° as numerically distant despite being physically adjacent. While this was successfully mitigated using phase unwrapping and Savitzky-Golay filtering in the post-processing pipeline, future iterations of this architecture could benefit from natively encoding track angles using sine and cosine continuous representations, effectively removing the discontinuity at the model level rather than relying on external smoothing.

The structural robustness of diffusion architectures comes with a well-known computational trade-off: generating a batch of trajectories requires executing the neural network hundreds or thousands of times sequentially. To proactively compensate for this iterative bottleneck, the proposed methodology purposefully engineered a highly efficient 1D U-Net. As detailed in the benchmarking results (Appendix B), generating a dataset of 1000 trajectories offline required 7.44 minutes. While this is perfectly feasible for simulation and airspace analysis, to the best of the author’s knowledge, no reference baselines currently provide explicit benchmarking data regarding generation latency for aircraft trajectories. Consequently, without a direct comparative baseline, it cannot be objectively established whether this generation time represents a practical limitation compared to existing data-driven methodologies or not.

Finally, a major practical finding during both the training of the DM and the execution of the evaluation pipeline was the absolute necessity of rigorous data normalization. Flight trajectories consist of heterogeneous variables operating on vastly different scales. For instance, altitude is typically measured in thousands of feet, groundspeed ranges between 50 and 150 knots, and track angle is bounded between 0 and 360 degrees. When early iterations of the training and evaluation scripts processed unscaled physical

units, the results were heavily skewed. Because standard loss functions (like Mean Absolute Error) treat all numerical errors equally, altitude deviations completely dominated the loss gradients. A small relative error in altitude (e.g., 500 feet) generated a massive penalty compared to a complete failure in predicting heading (e.g., 180 degrees). Consequently, the networks would aggressively optimize for altitude while ignoring the other kinematic variables, leading to poor convergence and artificially high evaluation errors. Implementing standard scaling techniques proved absolutely critical. Not only did normalization stabilize the diffusion process during training, but it also restored the sensitivity of the downstream evaluation metrics, allowing them to accurately penalize unrealistic dynamics across all variables equally.

Part IV

Closure

Conclusions

This thesis has demonstrated the feasibility and effectiveness of generating highly realistic, multi-modal kinematic flight trajectories using a 1D U-Net DM. By reframing trajectory generation as a continuous sequence-to-sequence diffusion process in the kinematic state space, the proposed model successfully captures both the complex spatial variance of different approach bundles and the fine-grained temporal dynamics of individual flights.

The choice of a 1D U-Net architecture over traditional autoregressive models (like RNNs or LSTMs) proved highly advantageous for trajectory generation. Autoregressive models generate sequences step-by-step, which inherently exposes them to compounding error accumulation over long sequences. In contrast, the DM processes the entire trajectory simultaneously as a 1D image. This global view inherently enforces structural constraints—ensuring that the trajectory seamlessly connects the initial approach point to the final touchdown on the runway—while the local convolutional kernels successfully maintain fine-grained kinematic consistency (such as steady descent rates).

Furthermore, the integration of FiLM conditioning proved capable of accurately preserving the empirical distributions of different aircraft types. To systematically evaluate these generated trajectories, this research established a comprehensive, multi-dimensional metric framework that simultaneously assesses spatial distribution (PCA, t-SNE, and Energy Distance), temporal fidelity (Discriminative Score), downstream predictive utility (TSTR), and data privacy (1-NN Memorization), all anchored by a novel self-diagnostic baseline. Utilizing this rigorous framework, the comparative analysis against the TimeGAN baseline confirmed that the proposed 1D U-Net DM achieves highly competitive state-of-the-art statistical performance, while fundamentally overcoming the training instability and mode collapse frequently observed in traditional GANs. Ultimately, this approach establishes a robust, kinematically realistic foundation for generating synthetic air traffic data, offering a powerful tool for airspace simulation, safety analysis, and future machine learning applications in aviation.

Future Work

The research presented in this thesis establishes a highly stable baseline for generating aircraft trajectories using a 1D U-Net Diffusion Model (DM). Crucially, by focusing on kinematic features rather than relying on spatial waypoints, this study demonstrates a streamlined, coordinate-agnostic model that intrinsically learns *how* to fly instead of simply memorizing *where* to fly. Building upon this baseline, several promising avenues for future research have been identified to further enhance the generation capabilities, physical realism, and operational applicability of the model.

8.1. Assessment of Zero-Shot Generation Capabilities

Because the proposed model learns the underlying principles of flight dynamics within a Kinematic State Space, it is theoretically capable of generating valid trajectories for unseen geographic regions. Future work should systematically assess the model’s zero-shot generation capabilities by evaluating its performance in novel Terminal Maneuvering Areas that were not included in the training dataset. This would conclusively demonstrate the model’s geographical independence and its ability to generalize complex terminal procedures with small or even zero retraining.

8.2. Integration of Physics-Informed Regularization

While this study has demonstrated the efficacy of a purely data-driven 1D Convolutional architecture in capturing temporal dynamics, future iterations could explore the integration of physical constraints using OpenAP data. Incorporating explicit aerodynamic regularization could potentially enhance the model’s strict adherence to performance envelopes, though it requires careful tuning of weighting parameters to ensure it does not constrain the model’s natural distributional learning. A thorough comparative analysis between a finely-tuned physics-informed model and the proposed baseline should be conducted to quantify the exact benefits of explicit aerodynamic guidance.

8.3. Validation with BlueSky Simulator

To further validate the realism and operational flyability of the synthesized trajectories, future research should integrate the model with a high-fidelity open-source air traffic simulator, such as BlueSky. Replaying the generated trajectories within the BlueSky simulation environment will allow for a rigorous, dynamic assessment of the model’s fidelity, ensuring that the synthesized paths strictly adhere to aerodynamic boundaries, aircraft performance limits, and practical operational constraints.

8.4. Integration of Weather Conditionals and Congestion Scenarios

Air traffic operations, particularly during landing and approach phases, are highly sensitive to meteorological conditions and air traffic density. While the current diffusion model codebase has already been architecturally prepared to accept continuous weather metadata (such as wind fields or temperature) as

conditioning parameters via the FiLM layers, the actual training, balancing, and exhaustive evaluation of these meteorological effects was left for future work. Fully integrating weather conditioning and rigorously analyzing its impact on the resulting flight dynamics represents a highly complex endeavor that could easily constitute an entire thesis by itself. Future research should leverage this existing architectural foundation to synthesize and study weather-driven edge-cases and “what-if” scenarios, such as wind-adjusted approach paths, holding patterns, or go-arounds caused by congestion or adverse weather.



Software User Guide & Repository Structure

This chapter details the structure of the software repository developed for this thesis and provides a user guide for running the implementation of the 1D U-Net Diffusion Model (DM). The code is organized into distinct modules, handling tasks from data preprocessing to model training and evaluation.

A.1. Code Repository Structure

The repository is modularly structured, dividing the workflow into data preparation scripts, core model definitions, and evaluation frameworks. A simplified overview of the primary directories is presented below:

- `data/`: Intended for storing raw and processed flight trajectory data.
- `scripts/`: Contains stand-alone Python scripts for data extraction, filtering, smoothing, clustering and visualization.
- `pi_ldm/`: The core module containing the architecture and training logic.
 - `src/`: Source code for dataset loading, physics loss formulation, DM training, sampling, and evaluation metrics.
 - `models/`: Directory designated for saving trained PyTorch model weights (e.g., `.pth` files).
 - `tests/`: Unit tests for verifying the software functionality.
 - `bin/`: Executables or helper scripts.
- `outputs/`: Stores generated synthetic trajectories, evaluation logs, and visualization plots.

The key dependencies for this project are: `torch` for deep learning and diffusion model implementation, `traffic` and `openap` for processing and analyzing aviation data, `numpy` and `pandas` for numerical data manipulation, `scikit-learn` and `tslearn` for trajectory clustering and evaluation metrics, and `geopandas`, `matplotlib`, and `seaborn` for geospatial mapping and visualizations.

A.2. Software User Guide

The workflow for using the developed software is divided into three main modules.

A.2.1. Module I: Data Loading, Preprocessing, Labeling & Clustering

This module handles the transformation of raw ADS-B data into a clean, normalized dataset suitable for model training. The scripts are located in the `scripts/` directory:

1. **Data Extraction (`extract_data.py`)**: Extracts raw trajectory segments from OpenSky.
2. **Data Filtering (`filter_data.py`, `filter_spatial.py`)**: Filters anomalies, incomplete flights, and restricts the spatial domain to the TMA.

3. **Clustering** (`trajectory_clusterer.py`): Groups trajectories into operational clusters (e.g., distinct approach routes).
4. **Data Smoothing** (`smooth_trajectories.py`): Applies smoothing algorithms (e.g., Savitzky-Golay) to reduce noise and ensure continuous derivatives.
5. **Dataset Building** (`dataset_builder.py`): Packages the finalized trajectories, computes scalars, and formats the data into normalized numpy arrays ready for model ingestion.

A.2.2. Module II: DM Model Training & Sampling

Once the data is processed, the Diffusion Model is trained using the `pi_ldm/src/` package:

1. **Training** (`train.py`): Instantiates the model using `model.py` and trains it on the prepared dataset loaded by `dataset.py`. Physics-informed constraints are optionally integrated via the `physics.py` module.
2. **Sampling** (`sample.py`): Generates new synthetic trajectories by passing Gaussian noise through the trained reverse diffusion process.

Trained checkpoints are automatically saved to the `pi_ldm/models/` directory.

A.2.3. Module III: Evaluation Framework - Diversity, Fidelity & Usefulness

The evaluation of the generated trajectories is handled by `pi_ldm/src/evaluation.py`. This module calculates all the metrics presented in Section 5.1.

Corresponding PCA, t-SNE and memorization plots are saved in the `outputs/` directory.

B

Benchmarking

This appendix provides an overview of the computational resources required to train and sample from the DM. Benchmarking the performance gives insight into the time and hardware requirements for deploying and extending this software. The values presented correspond to the training and generation of the full dataset of 1000 trajectories shown in Chapter 5.

B.1. Training Performance

The training of the Diffusion Model requires significant computational resources. It is for that reason that the models have been trained on Google Colab [55]. Table B.1 summarizes the hardware specifications and the average time required to complete the model’s training process.

Table B.1: Hardware specifications and training time benchmarking.

Parameter	Specification / Value
GPU Model	NVIDIA Tesla T4 (Google Colab)
VRAM	15 GB
RAM	12.7 GB (Colab)
DM Training Time	2.7 hours (656 epochs)

B.2. Sampling Times

Sampling new synthetic trajectories involves simulating the reverse diffusion process over a specified number of timesteps. The inference time is directly proportional to the number of denoising steps and the batch size. Table B.2 provides the sampling time required for the final 1000 trajectories generation.

Table B.2: Sampling time benchmarking for different batch sizes and denoising steps.

Batch Size	Denoising Steps	Average Sampling Time (min)
32	1000	7.44

References

- [1] Timoth  Krauth et al. “Deep Generative Modelling of Aircraft Trajectories in Terminal Maneuvering Areas”. In: *Machine Learning with Applications* 11 (Mar. 2023), p. 100446. DOI: 10.1016/j.mlwa.2022.100446. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2666827022001219> (visited on 02/15/2026).
- [2] Sebastiaan Wijnands et al. “Generation of Synthetic Aircraft Landing Trajectories Using Generative Adversarial Networks”. In: (Oct. 2024), p. 9. DOI: 10.5281/zenodo.14774664.
- [3] Olav Finne Praesteng Larsen et al. *Learning to Land Anywhere: Transferable Generative Models for Aircraft Trajectories*. Version 1. 2025. DOI: 10.48550/ARXIV.2511.04155. URL: <https://arxiv.org/abs/2511.04155> (visited on 02/15/2026). Pre-published.
- [4] Xavier Olive et al. “A Framework to Evaluate Aircraft Trajectory Generation Methods”. In: Oct. 2021. URL: https://www.researchgate.net/publication/354117181_A_Framework_to_Evaluate_Aircraft_Trajectory_Generation_Methods.
- [5] Abdulmajid Murad et al. “Synthetic Aircraft Trajectory Generation Using Time-Based VQ-VAE”. In: *2025 Integrated Communications, Navigation and Surveillance Conference (ICNS)*. Brussels, Belgium: IEEE, 2025, pp. 1–10. DOI: 10.1109/ICNS65417.2025.10976929. URL: <https://ieeexplore.ieee.org/document/10976929/> (visited on 02/15/2026).
- [6] Amy Hodgkin et al. *Conditioning Aircraft Trajectory Prediction on Meteorological Data with a Physics-Informed Machine Learning Approach*. Jan. 20, 2026. DOI: 10.48550/arXiv.2601.03152. arXiv: 2601.03152 [eess]. URL: <http://arxiv.org/abs/2601.03152> (visited on 02/23/2026). Pre-published.
- [7] Sundhar Vinodh Sangeetha et al. *Language Conditioning Improves Accuracy of Aircraft Goal Prediction in Untowered Airspace*. Sept. 17, 2025. DOI: 10.48550/arXiv.2509.14063. arXiv: 2509.14063 [cs]. URL: <http://arxiv.org/abs/2509.14063> (visited on 02/23/2026). Pre-published.
- [8] Timoth  Krauth et al. “Synthetic Aircraft Trajectories Generated with Multivariate Density Models”. In: *The 9th OpenSky Symposium*. OpenSky Symposium. MDPI, Dec. 30, 2021, p. 7. DOI: 10.3390/engproc2021013007. URL: <https://www.mdpi.com/2673-4591/13/1/7> (visited on 02/15/2026).
- [9] Zhisheng Xiao et al. *Tackling the Generative Learning Trilemma with Denoising Diffusion GANs*. 2022. arXiv: 2112.07804 [cs.LG]. URL: <https://arxiv.org/abs/2112.07804>.
- [10] Fernando Lucini. “The real deal about synthetic data”. In: *MIT Sloan Management Review* (2022). URL: <https://sloanreview.mit.edu/article/the-real-deal-about-synthetic-data/>.
- [11] Yingzhou Lu et al. “Machine learning for synthetic data generation: A review”. In: *arXiv preprint arXiv:2302.04062* (2023). URL: <https://arxiv.org/abs/2302.04062>.
- [12] Dina Kampouraki et al. *Techsonar report*. Tech. rep. European Data Protection Supervisor, 2020. URL: https://www.edps.europa.eu/data-protection/our-work/publications/reports/techsonar-2021-2022-report_en.

- [13] Leo L Pipino et al. “Data quality assessment”. In: *Communications of the ACM* 45.4 (2002), pp. 211–218. DOI: 10.1145/505248.506010. URL: <https://doi.org/10.1145/505248.506010>.
- [14] Valerie Sessions et al. “The effects of data quality on machine learning algorithms”. In: *Proceedings of the 11th International Conference on Information Quality (ICIQ)*. University of South Carolina, USA. 2006. URL: https://www.researchgate.net/publication/220918649_The_Effects_of_Data_Quality_on_Machine_Learning_Algorithms.
- [15] Rohit Babbar et al. “Data scarcity, robustness and extreme multi-label classification”. In: *Machine Learning* 108.8 (2019), pp. 1329–1351. URL: <https://link.springer.com/article/10.1007/s10994-019-05791-5>.
- [16] European Parliament and of the Council. *General data protection regulation (GDPR)*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [17] European University Institute of Florence. *The impact of the general data protection regulation (GDPR) on artificial intelligence*. Tech. rep. European Parliamentary Research Service, 2020. URL: <https://www.eurocontrol.int/publication/fly-ai-report>.
- [18] Daniel Carvajal-Patiño et al. “Synthetic data generation with deep generative models to enhance predictive tasks in trading strategies”. In: *Research in International Business and Finance* 62 (2022), p. 101747. DOI: <https://doi.org/10.1016/j.ribaf.2022.101747>. URL: <https://www.sciencedirect.com/science/article/pii/S0275531922001313>.
- [19] Bilgi Yilmaz et al. “Synthetic demand data generation for individual electricity consumers: Generative adversarial networks (GANs)”. In: *Energy and AI* 9 (2022), p. 100161. DOI: <https://doi.org/10.1016/j.egyai.2022.100161>. URL: <https://www.sciencedirect.com/science/article/pii/S2666546822000209>.
- [20] Marta Zaniolo et al. “FIND: A synthetic weather generator to control drought frequency, intensity, and duration”. In: *Environmental Modelling & Software* 172 (2024), p. 105927. URL: <https://www.sciencedirect.com/science/article/pii/S1364815223003134>.
- [21] Aston Zhang et al. *Dive into Deep Learning*. Cambridge University Press, 2023. URL: <https://d2l.ai>.
- [22] Eurocontrol. *Aviation data for research*. URL: <https://www.eurocontrol.int/dashboard/rnd-data-archive>.
- [23] Matthias Schäfer et al. “Bringing up OpenSky: A large-scale ADS-B sensor network for research”. In: *Proceedings of the 13th International Symposium on Information Processing in Sensor Networks (IPSN)*. IEEE Press. 2014, pp. 83–94. DOI: 10.1109/IPSN.2014.6846743.
- [24] Angela Nuic et al. “BADA: An advanced aircraft performance model for present and future ATM systems”. In: *International Journal of Adaptive Control and Signal Processing* 24.10 (2010), pp. 850–866. DOI: 10.1002/acs.1176. URL: https://www.researchgate.net/publication/229703718_BADA_An_advanced_aircraft_performance_model_for_present_and_future_ATM_systems.
- [25] Junzi Sun et al. “OpenAP: An open-source aircraft performance model for air transportation studies and simulations”. In: *Aerospace* 7.8 (2020). DOI: 10.3390/aerospace7080104. URL: <https://www.mdpi.com/2226-4310/7/8/104>.
- [26] MC Waller et al. “Considerations in the application of dynamic programming to optimal aircraft trajectory generation”. In: *IEEE Conference on Aerospace and Electronics*. IEEE. 1990, pp. 574–579. DOI: 10.1109/NAECON.1990.112828.

- [27] Daniel Delahaye et al. “Mathematical models for aircraft trajectory design: A survey”. In: *Air traffic management and systems*. Springer, 2014, pp. 205–247. DOI: 10.1007/978-4-431-54475-3_12.
- [28] Maryam Kamgarpour et al. “Trajectory generation for aircraft subject to dynamic weather uncertainty”. In: *49th IEEE Conference on Decision and Control (CDC)*. IEEE. 2010, pp. 2063–2068. DOI: 10.1109/CDC.2010.5717889.
- [29] Thomas Gräupl. “Validation of simulated synthetic air traffic against recorded air traffic in Germany”. In: *Integrated Communications Navigation and Surveillance (ICNS)*. IEEE. 2017, pp. 2063–2068. URL: <https://elib.dlr.de/111344/>.
- [30] Damien Jacquemart et al. “Conflict probability estimation between aircraft with dynamic importance splitting”. In: *Safety science* 51.1 (2013), pp. 94–100. DOI: 10.1016/j.ssci.2012.05.010.
- [31] Russell A. Paielli. *Empirical test of conflict probability estimation*. Tech. rep. NASA Ames Research Center, 1998. URL: <https://russp.us/CPetest.pdf>.
- [32] David Freedman. *Brownian motion*. Holden-Day, 1983, pp. 82–19573. DOI: 10.1007/978-1-4615-6574-1. URL: <https://books.google.nl/books?id=uxQGCAAAQBAJ&lpg=PA1&ots=7vei4rrC9Y&dq=brownian%20motion&lr&hl=ca&pg=PR5#v=onepage&q=brownian%20motion&f=false>.
- [33] Ian Goodfellow. “Nips 2016 tutorial: Generative adversarial networks”. In: (2016). arXiv: 1701.00160. URL: <http://arxiv.org/abs/1701.00160>.
- [34] Jonathon Shlens. “Notes on Kullback-Leibler Divergence and Likelihood”. In: (2014). DOI: 10.48550/arXiv.1404.2000. arXiv: 1404.2000 [cs.IT]. URL: <https://arxiv.org/abs/1404.2000>.
- [35] Ian Goodfellow et al. “Generative adversarial nets”. In: *Advances in neural information processing systems* 27 (2014).
- [36] Lintao Han et al. “Enhancing Remote Sensing Image Super-Resolution with Efficient Hybrid Conditional Diffusion Model”. In: *Remote Sensing* 15.13 (2023). DOI: 10.3390/rs15133452. URL: <https://www.mdpi.com/2072-4292/15/13/3452>.
- [37] Diederik P Kingma et al. “An introduction to variational autoencoders”. In: *Foundations and Trends® in Machine Learning* 12.4 (2019), pp. 307–392. DOI: 10.1561/22000000056. URL: <https://arxiv.org/abs/1906.02691>.
- [38] Arnault Motte et al. “Conditional Variational Autoencoders for Aircraft Type-Specific Trajectory Generation”. In: ().
- [39] Jonathan Ho et al. “Denoising diffusion probabilistic models”. In: *Advances in neural information processing systems* 33 (2020), pp. 6840–6851. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf.
- [40] Eric W Weisstein. “Normal distribution”. In: <https://mathworld.wolfram.com/> (2002).
- [41] Jiaming Song et al. *Denoising Diffusion Implicit Models*. 2022. DOI: 10.48550/arXiv.2010.02502. arXiv: 2010.02502 [cs.LG]. URL: <https://arxiv.org/abs/2010.02502>.
- [42] Ethan Perez et al. *FiLM: Visual Reasoning with a General Conditioning Layer*. 2017. DOI: 10.48550/arXiv.1709.07871. arXiv: 1709.07871 [cs.CV]. URL: <https://arxiv.org/abs/1709.07871>.

- [43] Vadim Sokolov et al. *Physics-Informed Diffusion Models for Vehicle Speed Trajectory Generation*. Feb. 4, 2026. DOI: 10.48550/arXiv.2602.05028. arXiv: 2602.05028 [stat]. URL: <http://arxiv.org/abs/2602.05028> (visited on 02/23/2026). Pre-published.
- [44] Leo Liberti et al. “Euclidean distance geometry and applications”. In: (2024), p. 64. DOI: 10.48550/arXiv.1205.0349. URL: <https://arxiv.org/abs/1205.0349>.
- [45] Douglas. Steinley. “K-means clustering: A half-century synthesis”. In: *British Journal of Mathematical and Statistical Psychology* 59.1 (2006), pp. 1–34. DOI: <https://doi.org/10.1348/000711005X48266>. eprint: <https://bpspsychub.onlinelibrary.wiley.com/doi/pdf/10.1348/000711005X48266>. URL: <https://bpspsychub.onlinelibrary.wiley.com/doi/abs/10.1348/000711005X48266>.
- [46] Zhi-Qiang Liu et al. “Bandpass Alignment from Sentinel-2 to Gaofen-1 ARD Products with UNet-Induced Tile-Adaptive Lookup Tables”. In: *Remote Sensing* 15 (May 2023), p. 2563. DOI: 10.3390/rs15102563.
- [47] Steffen Dereich et al. *Convergence rates for the Adam optimizer*. 2024. arXiv: 2407.21078 [math.OA]. URL: <https://arxiv.org/abs/2407.21078>.
- [48] N. Gallagher et al. “A theoretical analysis of the properties of median filters”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 29.6 (1981), pp. 1136–1141. DOI: 10.1109/TASSP.1981.1163708. URL: <https://doi.org/10.1109/TASSP.1981.1163708>.
- [49] Neal B. Gallagher. “Savitzky-Golay Smoothing and Differentiation Filter”. In: (). URL: <https://eigenvector.com/wp-content/uploads/2020/01/SavitzkyGolay.pdf>.
- [50] Fred B Bryant et al. “Principal-components analysis and exploratory and confirmatory factor analysis”. In: *Reading and understanding multivariate statistics*. American Psychological Association, 1995, pp. 99–136. URL: <https://psycnet.apa.org/record/1995-97110-004>.
- [51] Laurens Van der Maaten et al. “Visualizing data using t-SNE”. In: *Journal of machine learning research* 9.11 (2008). URL: <http://jmlr.org/papers/v9/vandermaaten08a.html>.
- [52] Gabor J Szekely et al. “Testing for equal distributions in high dimension”. In: *InterStat* 5.16.10 (2004).
- [53] Cristóbal Esteban et al. “Real-valued (medical) time series generation with recurrent conditional gans”. In: *arXiv preprint arXiv:1706.02633* (2017).
- [54] Jinsung Yoon et al. “Time-series generative adversarial networks”. In: *Advances in neural information processing systems* 32 (2019). URL: <https://papers.nips.cc/paper/2019/hash/c9efe5f26cd17ba6216bbe2a7d26d490-Abstract.html>.
- [55] Google Research. *Google Colab*. 2020. URL: <https://colab.research.google.com/>.